

Algoritmos y Estructuras de Datos 2

2025-01-01

Contents

1	Administración de Memoria	1
1.1	Breve introducción a sistemas operativos	1
1.1.1	El sistema operativo como API para los developers	1
1.1.2	El sistema operativo como administrador de los recursos de la máquina.	2
1.2	Definición de Administración de Memoria	2
1.2.1	Las direcciones de memoria	3
1.3	Evolución de la Administración de memoria	3
1.3.1	Ninguna abstracción de memoria	4
1.3.2	Espacios de direcciones	6
1.3.3	Memoria virtual	6
1.4	Administración de memoria a nivel de proceso	8
1.4.1	Secciones del memory layout	8
1.4.2	Punteros	17
1.4.3	Punteros en lenguajes manejados	26
1.5	Referencias adicionales	27
2	Análisis de algoritmos	29
2.1	Definición de Algoritmo	29
2.2	Análisis de Algoritmos	30
2.3	Análisis empírico de algoritmos	31
2.4	Análisis teórico de algoritmos	32
2.4.1	Mejor, peor y caso promedio	33
2.4.2	Análisis asintótico y notaciones comunes	34
2.4.3	Notación Big O, Big Omega y Big Theta	35
2.5	Determinar la complejidad de un algoritmo	38
2.5.1	Secuencia de instrucciones	40
2.5.2	If-Then-Else	40
2.5.3	Loops	40
2.5.4	Anidamiento	40
2.5.5	Complejidad logarítmica	41
2.5.6	Recursión	41
2.5.7	Más ejemplos	42

2.6	Revisitando los algoritmos y estructuras de datos	45
2.7	Referencias	45
3	Diseño de Algoritmos	47
3.1	Divide y conquista	47
3.1.1	Ejemplos de algoritmos de divide y conquista	48
3.2	Programación dinámica (DP)	50
3.2.1	Ejemplo de programación dinámica: Longest Common Subsequence (LCS)	52
3.3	Backtracking	53
3.3.1	Ejemplo: el problema de las N-reinas	54
3.4	Algoritmos Probabilísticos	54
3.4.1	Clasificación	55
3.4.2	Aleatoriedad	55
3.4.3	Algoritmos de Monte Carlo	56
3.4.4	Algoritmos de Las Vegas	56
3.4.5	Estructuras de datos probabilísticas	57
3.5	Algoritmos Genéticos	58
3.5.1	Definiciones esenciales	59
3.5.2	Representación de un individuo	59
3.5.3	Funcionamiento general	59
3.5.4	¿Cómo seleccionar los padres?	60
3.5.5	Introducir variabilidad	60
3.5.6	Recombinación	60
3.5.7	Mutación	60
3.6	Referencias	60
4	Algoritmos de búsqueda	61
4.1	Búsqueda secuencial	61
4.1.1	Implementación	62
4.1.2	Time complexity	62
4.1.3	Consideraciones importantes	62
4.2	Búsqueda binaria	62
4.2.1	Ejecución de búsqueda binaria	62
4.2.2	Implementación en Java	63
4.3	Búsqueda por interpolación	63
4.3.1	Referencias	64
4.4	Búsqueda por salto (Jump Search)	64
4.4.1	Complejidad	64
4.4.2	Referencias	64
4.5	Pathfinding	65
4.5.1	Pathfinding básico	65
4.5.2	Pathfinding basado en grafos	65
4.5.3	Referencias	67
5	Estructuras de almacenamiento externo	69

5.1	Conceptos esenciales	69
5.2	Discos Duros (Hard-disk drives)	69
5.2.1	Componentes	70
5.2.2	Funcionamiento	70
5.2.3	Tiempos de acceso	70
5.2.4	Tiempo de transferencia	71
5.2.5	Interfaces	71
5.3	Discos de Estado Sólido (SSD)	71
5.3.1	Componentes	72
5.3.2	Desempeño	72
5.4	Particionamiento de discos	72
5.4.1	Volumen	73
5.5	Sistemas de archivos	73
5.5.1	Propósito	73
5.6	Archivos	73
5.6.1	Estructura de un archivo	74
5.6.2	Tipos de archivos	75
5.7	Cache	75
5.7.1	Funcionamiento general	75
5.7.2	Operaciones en el cache	75
5.7.3	Tipos de cache a nivel general	76
5.7.4	Tipos de cache a nivel específicos	77
5.7.5	Problemas que afectan a todos los caches	77
6	Bases de Datos	79
6.1	Conceptos Clave	79
6.2	Tipos de Bases de Datos	79
6.3	Terminología Clave de las Bases de Datos	79
6.4	SQL (Structured Query Language)	80
6.5	Diseño de Bases de Datos	80
6.6	Beneficios de las Bases de Datos	80
6.7	Introducción a SQL	80
6.7.1	Operaciones Básicas en SQL	80
6.8	Introducción a NoSQL	82
6.8.1	Características de NoSQL	82
6.8.2	Tipos de Bases de Datos NoSQL	82
6.8.3	Casos de Uso de NoSQL	82
6.8.4	Consideraciones y Limitaciones	83
6.8.5	Ventajas de las bases de datos NoSQL	83

Chapter 1

Administración de Memoria

La administración de memoria conlleva diferentes ideas según el contexto. En el caso de los sistemas operativos, la administración de memoria se refiere a la gestión de la memoria principal de un sistema informático. En el caso del desarrollo de software, la administración de memoria se refiere a la gestión de la memoria de un programa en ejecución. Para lograr existosamente la segunda, se requiere comprender la primera.

En este capítulo, se abordarán los conceptos básicos de la administración de memoria en sistemas operativos y se entra en detalle a la administración de memoria en programas en ejecución.

1.1 Breve introducción a sistemas operativos

Capa de software que interactúa directamente con el hardware, intermediario entre el hardware que posee una computadora y las aplicaciones de software, facilitando la gestión de recursos del sistema.

1.1.1 El sistema operativo como API para los developers

API corresponde a las siglas de la palabra *Application Programming Interface*, concepto aplicable a varios contextos: API web, API del sistema operativo, API class libraries, etc; en nuestro caso, nos interesa el API del sistema operativo. Sea cual sea el contexto, la idea es la misma: *proveer una capa de abstracción para que los desarrolladores puedan interactuar con un sistema sin necesidad de conocer los detalles de su implementación.*

Un estándar para las API de los S.O es el **posix**. El posix son las siglas de la palabra en inglés *portable operative interface for unix*. Es un **estándar del IEEE para los APIS del S.O**.

Un ejemplo de API por ejemplo para abrir un archivo, se utiliza la función `fopen` que retorna un *pointer* o *handler* al archivo.

```
fptr = fopen("archivo.txt")
```

1.1.2 El sistema operativo como administrador de los recursos de la máquina.

El hardware es complejo. Si no hubiera un ente especializado en la administración de los recursos de la máquina, los desarrolladores tendrían que lidiar con la complejidad del hardware directamente. El sistema operativo se encarga de la administración de los recursos de la máquina, permitiendo a los desarrolladores interactuar con la máquina de forma más sencilla. *El sistema operativo, hace transparentes los recursos de la máquina.*

Los recursos administrados son:

- CPU
- Memoria Principal
- Discos
- I/O

Thread vs Process

Un programa es un archivo ejecutable que contiene el código o el conjunto de instrucciones del procesador, que se almacena como un archivo en el disco. Cuando un programa se ejecuta, se carga en memoria y se convierte en un **proceso**. Un proceso activo incluye los recursos administrados por el sistema operativo que el programa necesita para ejecutarse: registros de procesador, contadores de programas, punteros de pila, páginas de memoria, etc.

Un **thread (hilo)** es un flujo de control dentro de un proceso, y cada hilo tiene su propia pila de llamadas, registros de procesador y estado de ejecución. Múltiples hilos comparten el mismo espacio de direcciones de memoria, archivos abiertos, etc.

1.2 Definición de Administración de Memoria

Es el conjunto de tareas que realiza el sistema operativo para gestionar la memoria principal de una computadora. La administración de memoria es una parte fundamental de los sistemas operativos modernos, ya que permite a los programas ejecutarse sin tener que preocuparse por la gestión de la memoria.

Algunas de las responsabilidades de la administración de memoria son:

- Asignar/Liberar memoria al inicio/fin de un proceso.
- Controlar la memoria asignada a un proceso.
- Minimizar la fragmentación.

- Mantener la integridad de los datos.

La idea de controlar la memoria asignada a un proceso, tiene como fin hacer que el mismo tenga un límite y aislamiento, para que así no afecte a otros procesos en la memoria que se estén ejecutando.

1.2.1 Las direcciones de memoria

Antes de continuar con el resto de este capítulo, es clave entender el concepto de direcciones de memoria. Para esto utilizaremos un cotidiano.

Imagine un edificio de apartamentos, en el que todos los apartamentos son exactamente del mismo tamaño y están numerados de manera consecutiva. Cada apartamento únicamente puede alojar a una sola persona. Una familia por lo tanto, ocupará varios apartamentos contiguos. El edificio puede continuar en creciendo con el tiempo conforme se construyan nuevos pisos, pero la constructora tendrá un máximo de apartamentos que puede construir. La recepción del edificio puede llevar correspondencia a cualquier apartamento, incluso cuando se construyan nuevos apartamentos, pero siempre limitado por el número de apartamentos que se pueden construir y por el número de apartamentos que ya están contruídos.

Aplicando el ejemplo a la computadora, un sistema operativo (*la recepción*) tiene la capacidad de generar direcciones de memoria de n bits (usualmente 32 o 64 bits). Por ejemplo, si es son direcciones de 32 bits, el sistema operativo puede generar 2^{32} direcciones de memoria, desde la dirección 0 hasta la dirección $2^{32} - 1$. Cada dirección de memoria corresponde a una casilla (*un apartamento*) que puede alojar 1 byte (*una persona*). Las variables (*familias*) puede ser de 1 o más bytes, siempre contiguos.

Si el usuario (*la constructora*) instala más RAM, el sistema operativo podrá accederlo hasta el máximo que sus direcciones lo permitan. Por ejemplo, si el sistema operativo tiene direcciones de 32 bits, podrá acceder a 2^{32} bytes de memoria, es decir 4GB. Si el usuario instala 8GB de RAM, el sistema operativo solo podrá acceder a 4GB, ya que no tiene direcciones para más.

Suponiendo un sistema operativo con direcciones de 16 bits, el sistema operativo podrá acceder a 2^{16} bytes de memoria, es decir 64KB. El rango de direcciones sería de 0 a 65535.

1.3 Evolución de la Administración de memoria

La memoria ha sido un componente fundamental en las computadoras desde sus inicios y por ende ha necesitado del sistema operativo para administrarla. Conforme los recursos computacionales se adaptan a las necesidades de los usuarios y las prestaciones de hardware, la administración de memoria se adapta para reducir el *overhead* y mejorar la eficiencia.

A continuación se presenta la evolución de la abstracción de la memoria en los sistemas operativos.

1.3.1 Ninguna abstracción de memoria

La abstracción más simple es no tener ninguna. Las computadoras mainframe de los años 60, minicomputadoras de los 70s y computadoras personales en los 80s, no tenían abstracción de memoria. Los programas accedían directamente a la memoria **física**. Un programa con la instrucción `MOV REGISTER1, 1000` en realidad accedía a la dirección de memoria 1000.

Había cierta organización de la memoria:

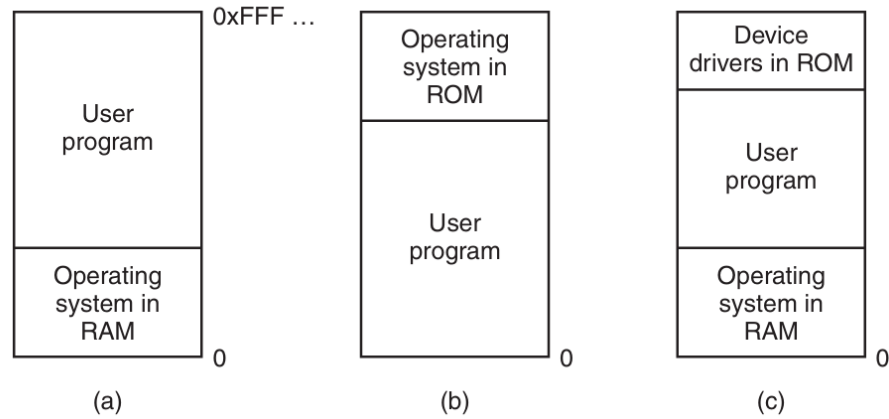


Figure 1.1: Organización de la memoria sin abstracción

Bajo estas condiciones, era imposible ejecutar más de un programa a la vez (*multiprogramación*), ya que cada programa accedía directamente a la memoria física. Dos o más programas cargados en memoria, podían afectarse entre sí causando errores en la ejecución. El uso de threads era posible dado que compartían la misma memoria, pero de poco valor.

El *swapping* se introduce como técnica para “pausar” la ejecución de un, guardar su estado en disco y cargar otro programa en memoria. Esta técnica permitió la multiprogramación *a cierto grado*, dado que solo un programa podía estar cargado en memoria en un momento dado. La idea era que mientras un programa estaba esperando por una operación de I/O, otro programa podía ejecutarse.

La computadora *IBM 360*, introduce una técnica para poder tener dos programas en memoria: *static relocation*. Cuando un programa se cargaba en memoria, se le asignaba una dirección base (la dirección inicial en la que se carga) y se modificaban todas las referencias a memoria para sumarles la dirección base.

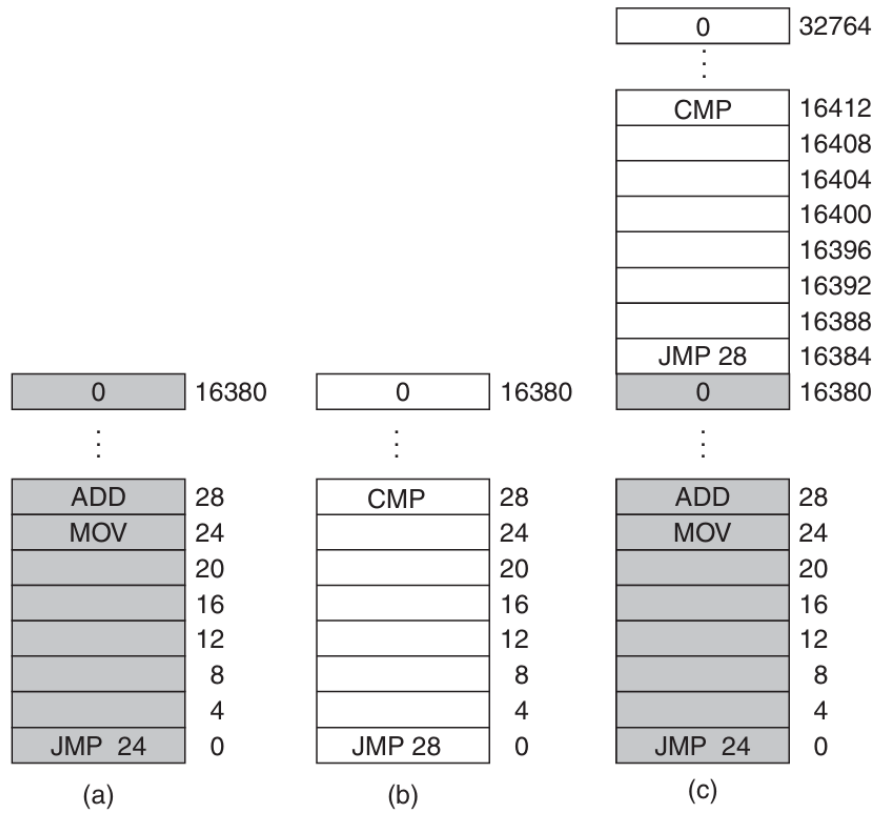


Figure 1.2: El problema de la re-ubicación

Esto funcionaba pero claramente no era eficiente puesto que entre más grande fuera el programa, más tarda en cargarse.

Multiprogramación

Se denomina multiprogramación a una técnica por la que dos o más procesos pueden alojarse en la memoria principal y ser ejecutados concurrentemente por el procesador o CPU. [...] la ejecución de los procesos (o hilos) se va solapando en el tiempo a tal velocidad, que causa la impresión de realizarse en paralelo (simultáneamente)

[...] En los antiguos sistemas monoprogramados, cuando un proceso en ejecución requería hacer uso de un dispositivo de E/S, el procesador quedaba ocioso mientras el proceso permaneciese en espera y no retomara su ejecución *de Wikipedia*

1.3.2 Espacios de direcciones

Constituye una abstracción para la memoria física. Cada programa tiene su propio espacio de direcciones, que va desde 0 a un valor máximo. El sistema operativo se encarga de mapear las direcciones virtuales a direcciones físicas. Dos programas pueden ver la dirección 28, pero en realidad se refieren a direcciones físicas diferentes.

Bajo este enfoque, el hardware provee dos registros llamados *base* y *límite*. Cuando un programa específico se ejecuta, el sistema operativo carga el *base* y *límite* con los valores correspondientes al espacio de direcciones del programa. Cada vez que el programa accede a una dirección de memoria, el hardware verifica que la dirección esté dentro del rango permitido por el *base* y *límite* y genera la dirección física correspondiente.

Una limitante de este enfoque es que el programa completo debe caber en la memoria, lo cual es claramente poco práctico para los programas modernos con requerimientos de memoria cada vez más agresivos y que compiten con muchos otros programas ejecutándose concurrentemente.

Concurrencia vs Paralelismo

La concurrencia se refiere a la capacidad de un sistema de llevar a cabo múltiples tareas en un mismo periodo de tiempo traslapándose entre sí, pero no implica que se ejecuten a la misma vez. En paralelismo, las tareas se ejecutan al mismo tiempo. Paralelismo implica múltiples núcleos de procesamiento. La concurrencia puede lograrse con un solo núcleo bajo multiprogramación.

1.3.3 Memoria virtual

La memoria virtual es la solución para poder ejecutar programas que no caben en la memoria física. La idea básica es que cada programa tiene su propio espacio

de direcciones dividido en bloques llamados *páginas*. Cada página es un rango contiguo de direcciones.

Las páginas se mapean a bloques de memoria física llamados *frames*, pero no todas las páginas necesitan estar cargadas. Cuando un programa referencia una página que está cargada en memoria (*page-hit*), el hardware mapea la dirección virtual a la dirección física correspondiente. Si la página no está cargada, el sistema operativo la carga desde el disco a un frame libre en memoria y re-ejecuta la instrucción que causó el fallo de página (*page-fault*).

Entonces, cuando un programa tiene una instrucción `MOV REGISTER1, 1000`, la dirección 1000 es una dirección virtual parte de su espacio de direcciones virtuales.

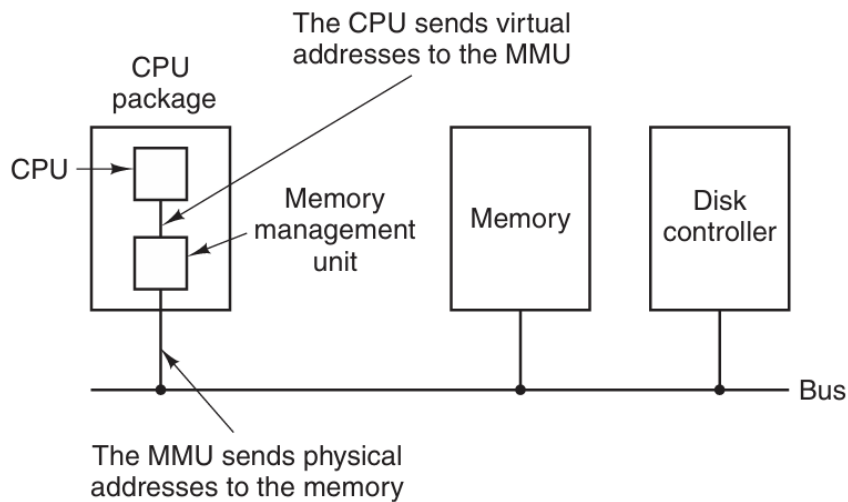


Figure 1.3: Funcionamiento del MMU

Como se nota en la imagen anterior, hay hardware especializado, el *Memory Management Unit (MMU)* que se encarga de mapear las direcciones virtuales a direcciones físicas. El MMU tiene una tabla de páginas que mapea las direcciones virtuales a direcciones físicas. La tabla de páginas se mantiene en memoria y el MMU la consulta cada vez que necesita mapear una dirección virtual a una dirección física.

El MMU tiene una tabla que lleva el inventario de páginas cargadas y su correspondiente frame. Dicha tabla aloja información estadística sobre las páginas, como la frecuencia de uso, para poder tomar decisiones sobre qué páginas mantener en memoria y cuáles sacar. Dado que los frames son limitados, se utilizan algoritmos de reemplazo de páginas para decidir cuál página sacar de memoria cuando se necesita cargar una nueva y no hay espacio.

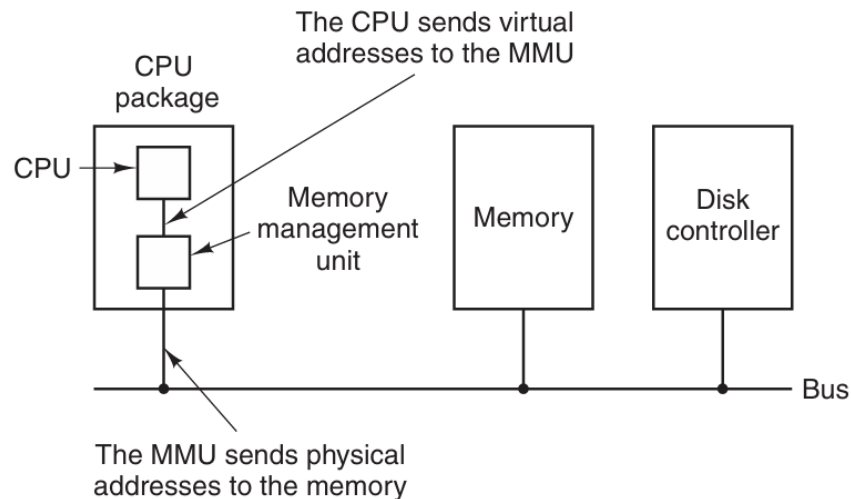


Figure 1.4: Relación entre direcciones virtuales y físicas

1.4 Administración de memoria a nivel de proceso

Para un proceso (programa en ejecución con recursos asignados), la administración de memoria se refiere a la gestión de la memoria asignada. La abstracción utilizada por el sistema operativo, es transparente para el proceso, el cuál únicamente utiliza los mecanismos disponibles para manipular la memoria.

Dependiendo del lenguaje y compilador, el layout de memoria puede variar. Un programa en C, usualmente tiene el siguiente layout de memoria:

A continuación se describen a mayor detalle cada una de estas partes.

Se recomienda leer este artículo de [GeeksForGeeks](#) con detalles sumamente relevantes del memory layout

1.4.1 Secciones del memory layout

1.4.1.1 Text

Contiene el código **ejecutable**. Usualmente es compartido entre procesos de un mismo programa. Es de solo lectura.

1.4.1.2 Initialized Data

Llamado también el segmento de datos. Contiene las variables **globales y/o estáticas** inicializadas. Es de lectura y escritura. Tiene dos áreas: una para

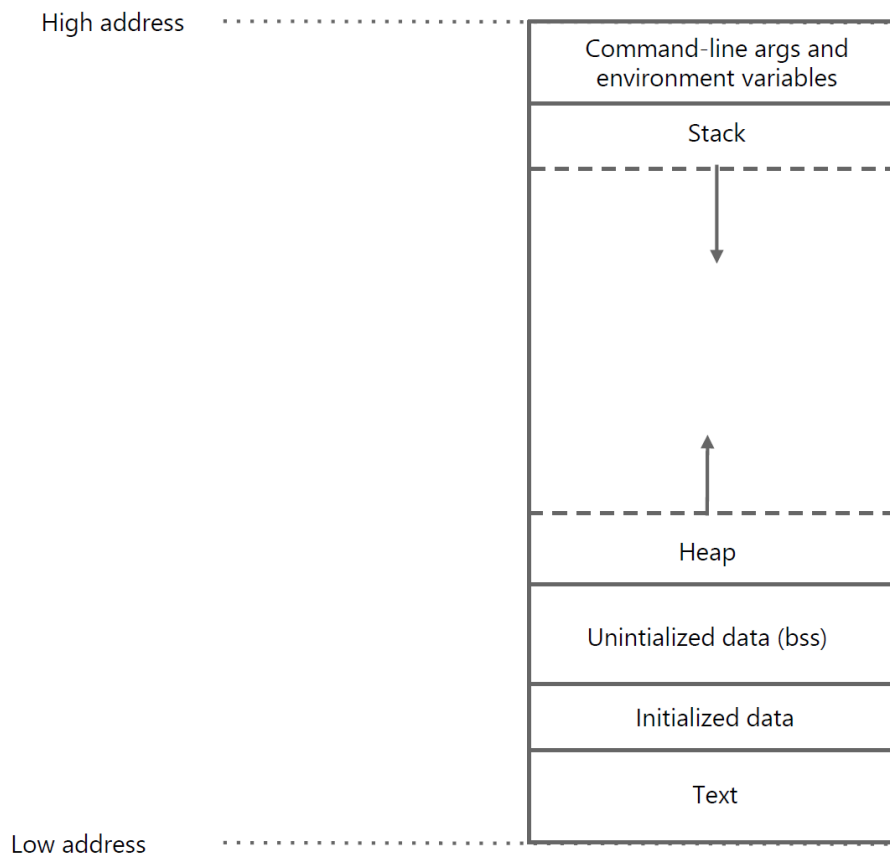


Figure 1.5: Layout de memoria de un programa en C

variables read-only y otra para variables read-write. Por ejemplo

```
int x = 10; //read-write
const int y = 20; //read-only
char s[] = "hola"; //read-write
```

```
int main() {
    // Código principal
}
```

¿static en C? Al usarlo sobre variables que están dentro de una función, permite que el valor de las mismas persista entre llamadas. Al usarlo sobre funciones o variables de ámbito global, garantiza que dicho elemento sólo exista en la unidad de compilación en la que se encuentre declarado. *fente: stackoverflow*

1.4.1.3 Uninitialized Data Segment

Usualmente llamado el segmento bss (block started by symbol). Contiene las variables **globales y/o estáticas** no inicializadas. Es de lectura y escritura. Los datos en este segmento, son inicializados en cero por el compilador antes de que el programa empiece a ejecutarse.

Por ejemplo,

```
int x; //uninitialized
int y; //uninitialized

int main() {
    // Código principal

    static int z; //uninitialized
}
```

1.4.1.4 Command-line arguments and environment variables

Esta sección de la memoria se carga con los argumentos de la línea de comandos y las variables de entorno. Por *argumentos de la línea de comandos* se entiende los argumentos que se pasan al programa al momento de ejecutarlo. Por ejemplo, `./programa -a -b -c_`. Estos argumentos se pueden acceder programáticamente mediante:

```
int main(int argc, char* argv[]) {
    // argc es el número de argumentos
    // argv es un arreglo de strings con los argumentos
    // argv[0] es el nombre del programa
    // argv[1] es el primer argumento
```



```
// argv[2] es el segundo argumento

// Ejemplo
printf("El primer argumento es %s\n", argv[1]);
}
```

Las *variables de entorno* son variables que se definen en el sistema operativo y que pueden ser accedidas por los programas. Cuando se ejecuta un programa en la terminal, se pueden definir variables de entorno. Por ejemplo (en bash), `export MYVAR="Ejemplo de variable"`. Estas variables se pueden acceder programáticamente mediante la variable global `environ`.

```
extern char** environ;

int main() {
    // Iterar sobre las variables de entorno
    for (int i = 0; environ[i] != NULL; i++) {
        printf("%s\n", environ[i]);
    }
}
```

1.4.1.5 Stack

Es una sección de la memoria que se utiliza para almacenar las variables locales de las funciones y los argumentos de las mismas. Es de tamaño fijo y se expande y contrae dinámicamente. Utilizar el stack es transparente (e inevitable) para el programador, por lo que el manejo de la memoria es menos propenso a errores.

El stack es *LIFO* (Last In, First Out). Cada vez que se llama a una función, se crea un *stack frame* que contiene las variables locales de la función, los argumentos y el *return address* (la dirección a la que se debe regresar una vez que la función termina). Cuando dicha función termina, el *stack frame* se elimina (se marca la memoria como libre). Es por tal razón que las variables locales se les llama *automáticas*.

Las variables locales almacenables en el stack deben ser de tamaño conocido al momento de la compilación. Por esta razón, memoria dinámica como listas enlazadas no puede almacenarse en el stack.

1.4.1.6 Heap

Es una sección de la memoria que se utiliza para almacenar datos que no tienen un tamaño conocido al momento de la compilación y cuyo tiempo de vida es controlado por el programador. Por ejemplo, listas enlazadas, árboles, etc. El heap es de tamaño variable y se expande y contrae dinámicamente. El programador es responsable de manipular la memoria en el heap, es decir, asignar, des-asignar y re-dimensionarla.

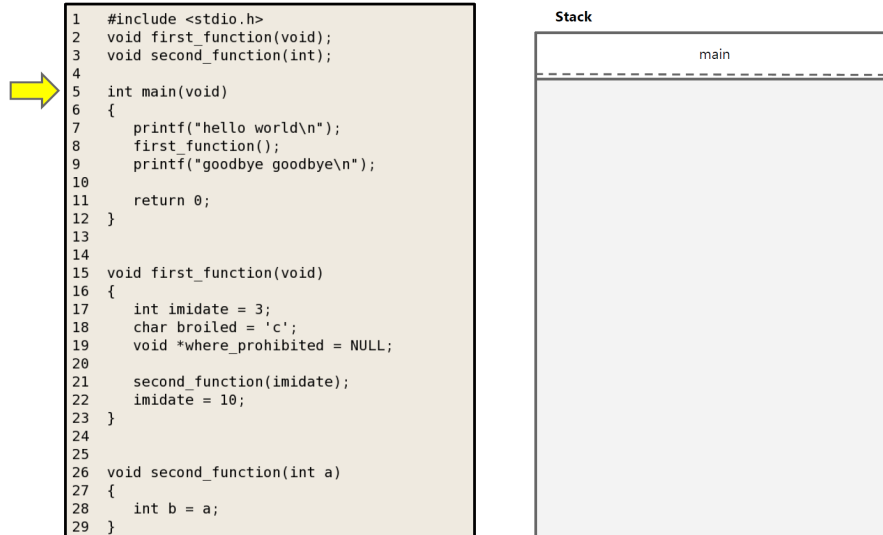


Figure 1.6: Visualización del stack (1 de 5)

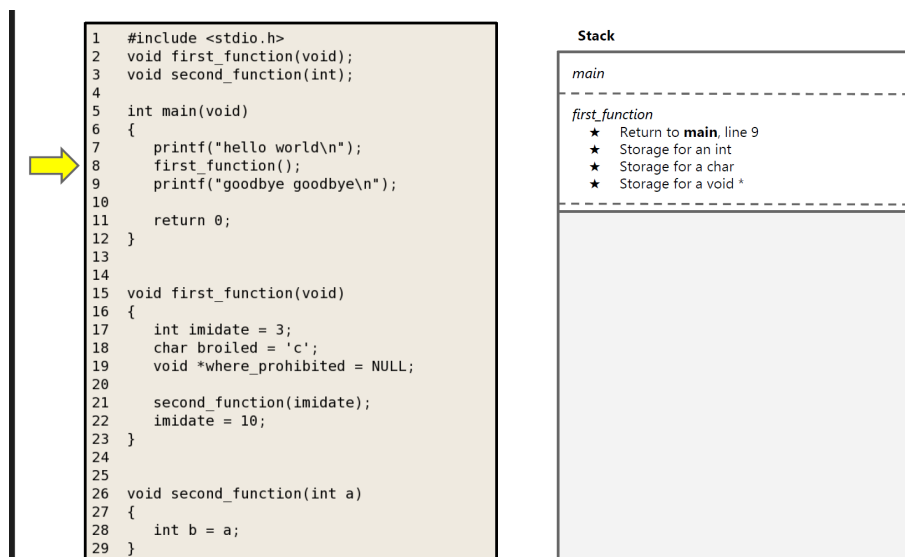


Figure 1.7: Visualización del stack (2 de 5)

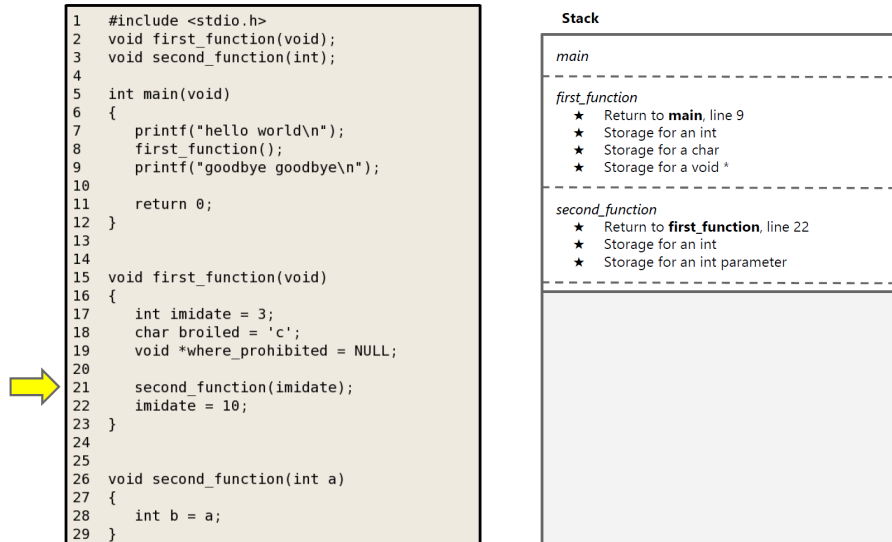


Figure 1.8: Visualización del stack (3 de 5)

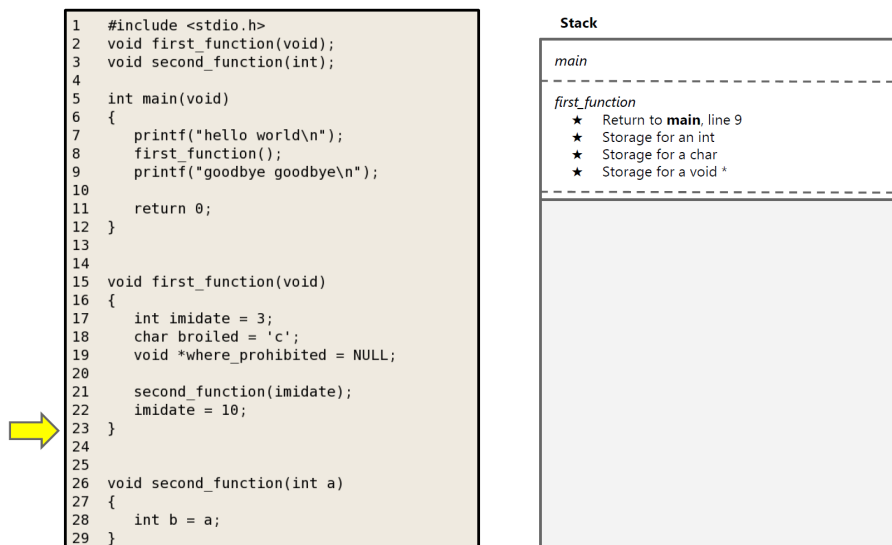


Figure 1.9: Visualización del stack (4 de 5)

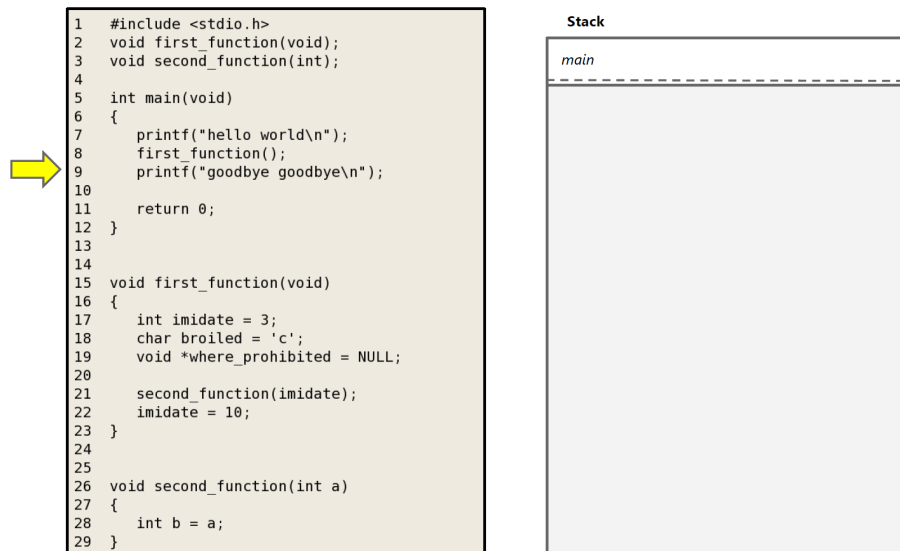


Figure 1.10: Visualización del stack (5 de 5)

Para interactuar con la memoria, el programador utiliza funciones como `malloc`, `free`, `realloc`, `calloc` y `new/delete` (en C++). Estas funciones conforman el API del heap en C/C++.

¿Es posible evitar usar el Heap?

Sí, es posible evitar usar el heap. Sin embargo, esto implica que el programador únicamente podrá utilizar variables de tamaño conocido en tiempo de compilación, limitando la flexibilidad y el alcance de lo que el programa puede realizar.

El siguiente código muestra un ejemplo de cómo se puede utilizar el heap en C:

¿Qué pasa si no se libera la memoria en el heap?

Si no se libera la memoria en el heap, se produce una fuga de memoria (*memory leak*). Esto significa que la memoria asignada al programa no se libera y se pierde. Con el tiempo, el programa puede quedarse sin memoria y fallar.

En los ejemplos de código anteriores se utilizan punteros, por ejemplo `int* age = malloc(sizeof(int))`. En la siguiente sección se abordarán los punteros en mayor detalle.

1.4.1.7 Heap vs Stack

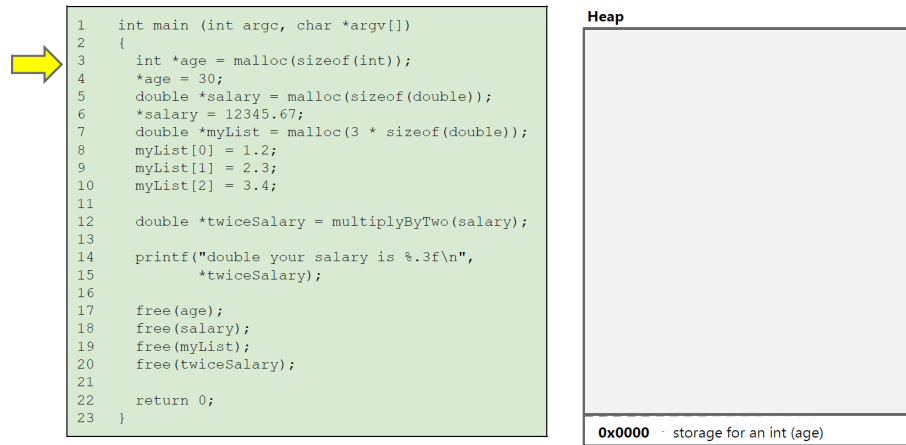


Figure 1.11: Visualización del heap (1 de 4)

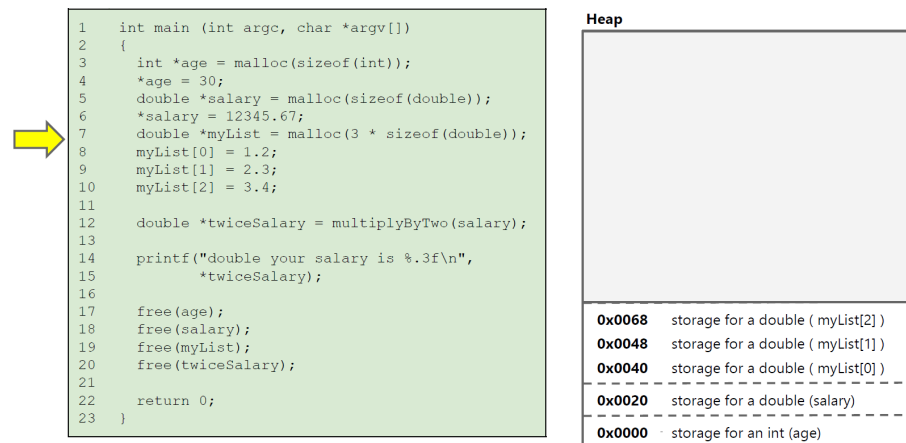


Figure 1.12: Visualización del heap (2 de 4)

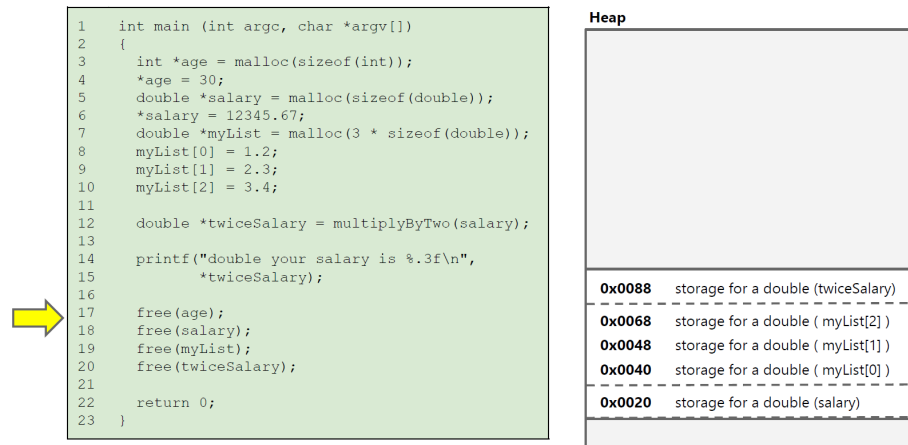


Figure 1.13: Visualización del heap (3 de 4)

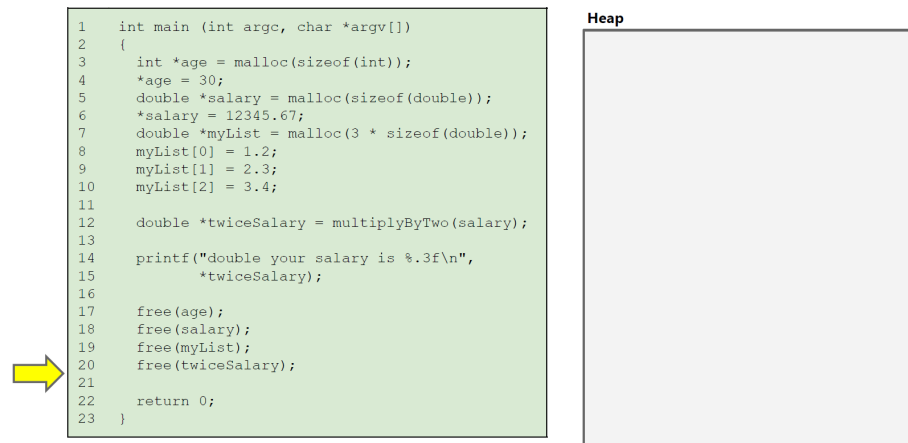


Figure 1.14: Visualización del heap (4 de 4)

Heap	Stack
Memoria dinámica	Memoria estática
Tamaño variable	Tamaño fijo
Programador es responsable de la memoria	Programador no es responsable de la memoria
No es transparente	Transparente
Propenso a errores (bugs introducidos por el programador)	Menos propenso a errores (bugs del sistema operativo)

1.4.2 Punteros

Es un tipo de datos especial definido como parte del API del Heap. Al ser un tipo de datos, define un rango posible de valores que puede contener y un conjunto de operaciones que soporta.

- Rango de valores de un puntero: direcciones de memoria
- Operaciones soportadas: asignación, des-asignación, aritmética de punteros, acceso a memoria

Para declarar un puntero se utiliza código similar al siguiente:

```
int* ptr = malloc(sizeof(int));
char* cptr = malloc(sizeof(char));
double* dptr = malloc(sizeof(double));
MyClass* myptr = new MyClass(); // En C++, dado que C no soporta clases
void* vptr = malloc(100);

// Si NULL no estuviera definido, se podría usar 0 o definirlo manualmente
int* nPtr = NULL;

// 0 es el único valor literal que se puede asignar a un puntero en C.
// Equivalente a NULL
int* nPtr2 = 0;
int* nPtr3 = nullptr; // En C++ 14
```

Si visualizamos la memoria como una tabla con columnas y filas, para el código anterior tendríamos:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	ptr	0x64	4B	Stack	Pointer
0x4	cptr	0x68	4B	Stack	Pointer
0x8	dptr	0x69	4B	Stack	Pointer
0x12	myptr	0x72	4B	Stack	Pointer
0x16	vptr	0x92	4B	Stack	Pointer
0x1A	nPtr	0x0	4B	Stack	Pointer
0x1E	nPtr2	0x0	4B	Stack	Pointer

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x22	nPtr3	0x0	4B	Stack	Pointer
0x64		0	4B	Heap	int
0x68		"	1B	Heap	char
0x69		0	8B	Heap	double
0x72		0	20B	Heap	MyClass
0x92		0	100B	Heap	?

Las direcciones de la tabla son ficticias y no corresponden a direcciones reales de memoria. Asumimos que el heap empieza en la dirección hexadecimal 64. Asumimos que cada direcciones de 32 bits, es decir 4 bytes. Asumimos que la clase `MyClass` tiene un tamaño de 20 bytes.

Como se puede notar, para acceder a la memoria, se necesita dos componentes: la llamada al API (en este caso `malloc`) y un puntero para poder acceder a la memoria creada por el API. El puntero siempre estará en el stack, y es una variable automática como cualquier otra. Sin embargo, al liberarse junto con el frame, la memoria en el Heap no se libera.

Una llamada a `malloc` sin asignar un puntero, por ejemplo

```
malloc(sizeof(int));
```

Resulta en una tabla de memoria como:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x64		0	4B	Heap	int

Pero al no haber ningún pointer en el stack que almacene la dirección `0x64`, dicha memoria es inaccesible y se produce una fuga de memoria.

No hay nada “mágico” con respecto a los punteros. Son simplemente un tipo de dato como cualquier otro. La diferencia es que los punteros contienen direcciones de memoria en lugar de valores.

¿Cuál es el proposito de declarar pointers con tipo si todos ocupen el mismo espacio?

Type check: El compilador pueda hacer type checking. Por ejemplo, si se declara un puntero de tipo `int`, el compilador no permitirá asignarle una dirección de memoria de un `char`.

Read/Write size: El compilador sabe cuántos bytes leer o escribir al acceder a la memoria a través de un puntero.

Aritmética de pointers: El compilador sabe cuántos bytes sumar o restar al hacer aritmética de punteros.

A continuación se describen las operaciones más comunes con punteros.

1.4.2.1 Operador Address-Of (&)

El operador **unario** **Address-Of**, designado por **&** (no confundir con el operador binario **&** para boolean) se utiliza para obtener la dirección de memoria de una variable. Por ejemplo, si se tiene una variable `int x = 10`, se puede obtener la dirección de memoria de `x` mediante `&x`. Se puede aplicar a memoria en el stack o en el heap.

```
int x = 10;
int* ptr = &x;
```

Para el código anterior, la tabla de memoria sería:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	10	4B	Stack	int
0x4	ptr	0x0	4B	Stack	Pointer

Considere el siguiente código:

```
#include <iostream>
using namespace std;

int main()
{
    int x = 20;

    // Pointer pointing towards x
    int* ptr = &x;

    cout << "The address of the variable x is :- " << ptr;
    return 0;
}
```

Dicho programa generará la siguiente salida (espacio de direcciones de 48b):

The address of the variable x is: 0x7ffbf7b3b7c

1.4.2.2 Operador de indirección/de-referencia (*)

El operador **unario** **de-referencia**, designado por ***** (no confundir con el operador binario *****), se utiliza para acceder al valor almacenado en la dirección de memoria apuntada por un puntero. Por ejemplo, si se tiene un puntero `int* ptr` que apunta a la dirección de memoria de una variable `x`, se puede acceder al valor de `x` mediante `*ptr`.

Por ejemplo, el siguiente código:

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    int x = 3899;
    int* price;

    price = &x;

    cout << "The address of x is : " << &x;
    cout << "The value of price : " << price;
    cout << "The value stored at the variable pointed by price is: " << (*price);
    cout << "The value x is: " << x;
    return 0;
}
```

Genera la siguiente salida:

```
The address of x is : 0x7ffffb7b3b7c
The value of price : 0x7ffffb7b3b7c
The value stored at the variable pointed by price is: 3899
The value x is: 3899
```

El operador de indirección también sirve para modificar el valor de la variable apuntada por el puntero. Por ejemplo, el siguiente código:

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    int x = 3899;
    int* price;

    price = &x;

    cout << "The value of x is: " << x << endl;
    cout << "The value of price is: " << *price << endl;

    *price = 1000;

    cout << "The value of x is: " << x << endl;
    cout << "The value of price is: " << *price << endl;

    return 0;
}
```

Genera la siguiente salida:

```
The value of x is: 3899
The value of price is: 3899
The value of x is: 1000
The value of price is: 1000
```

La tabla de memoria para el código anterior sería inicialmente:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	3899	4B	Stack	int
0x4	price	0x0	4B	Stack	Pointer

y luego de la instrucción `*price = 1000;`,

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	1000	4B	Stack	int
0x4	price	0x0	4B	Stack	Pointer

1.4.2.3 Sharing

Es un concepto que se refiere a la posibilidad de tener dos o más punteros que apunten a la misma dirección de memoria. Por ejemplo, si se tiene un puntero `int* ptr` que apunta a la dirección de memoria de una variable `x`, se puede tener otro puntero `int* ptr2` que apunte a la misma dirección de memoria de `x`.

```
int x = 20;
int* ptr = &x;
int* ptr2 = ptr;
```

La tabla de memoria para el código anterior sería:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	10	4B	Stack	int
0x4	ptr	0x0	4B	Stack	Pointer
0x8	ptr2	0x0	4B	Stack	Pointer

Mediante cualquiera de los punteros `ptr` o `ptr2`, se puede leer/modificar el valor de `x`. Por ejemplo:

```
int x = 20;
int* ptr = &x;
int* ptr2 = ptr;
```

```

*ptr = 30;
cout << x; // Imprime 30
cout << *ptr2; // Imprime 30
cout << *ptr; // Imprime 30

```

1.4.2.4 Shallow-copy vs Deep-copy

Shallow-copy implica copiar el puntero, pero no el valor al que apunta. Por ejemplo, para el siguiente código:

```

int x = 20;
int* ptr = &x;
int* ptr2 = ptr;

```

La tabla de memoria para el código anterior sería:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	10	4B	Stack	int
0x4	ptr	0x0	4B	Stack	Pointer
0x8	ptr2	0x0	4B	Stack	Pointer

Nótese que solo hay un espacio con el valor 10. Ambos pointers “apuntan” a la misma dirección. Se pueden crear n copias de punteros, pero solo habrá un espacio de memoria al que todos apuntan.

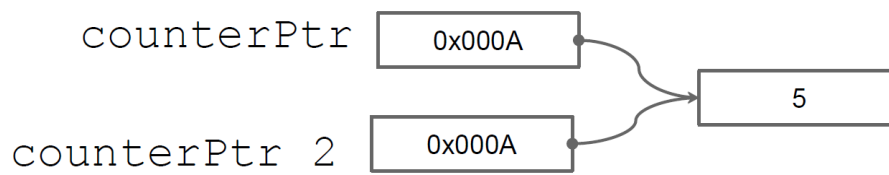


Figure 1.15: Shallow copy

Por otro lado, *Deep-copy* implica copiar el valor al que apunta el puntero. Por ejemplo, el siguiente código:

```

int x = 20;
int* ptr = &x;
int* ptr2 = malloc(sizeof(int));
*ptr2 = *ptr;

```

En este caso, la tabla de memoria sería:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	20	4B	Stack	int
0x4	ptr	0x0	4B	Stack	Pointer
0x8	ptr2	0x64	4B	Stack	Pointer
0x64		20	4B	Heap	int

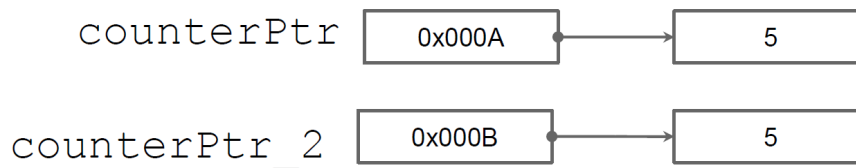


Figure 1.16: Deep copy

Deep-copy puede implicar más trabajo que un simple `malloc`. Por ejemplo, al copiar una estructura de datos a otra, si alguno de los campos de la estructura es un puntero, se debe copiar el valor al que apunta el puntero, no el puntero en sí.

1.4.2.5 Aritmética de punteros

Permite sumar o restar un número entero a un puntero. La aritmética de punteros es útil para acceder a elementos de un array o para moverse a través de una estructura de datos. Por ejemplo, considere el siguiente código:

```

int arr[5] = {10, 20, 30, 40, 50};
int* ptr = arr;

cout << *ptr; // Imprime 10
cout << *(ptr + 1); // Imprime 20
cout << *(ptr + 2); // Imprime 30

```

En este caso, `ptr` apunta al primer elemento del array `arr`. Al sumar 1 a `ptr`, se mueve al siguiente **elemento** del array. Al sumar 2 a `ptr`, se mueve dos elementos hacia adelante. Nótese que la aritmética de punteros es en términos de elementos, no de bytes. Dado que el pointer tiene un tipo, el compilador sabe cuántos bytes sumar o restar al hacer aritmética de punteros. El array `arr` ocupa 50 bytes contiguos en memoria y cada elemento ocupa 4 bytes. Por lo tanto, `ptr + 1` salta de 4 en 4 bytes.

Para efectos de arrays en C/C++, `arr[i]` es equivalente a `*(arr + i)`. Por ejemplo, el siguiente código:

```

int arr[5] = {10, 20, 30, 40, 50};

cout << arr[0]; // Imprime 10

```

```
cout << arr[1]; // Imprime 20
cout << arr[2]; // Imprime 30
```

Dado que acceder cualquier elemento de un array es equivalente a acceder a la dirección de memoria del primer elemento y sumarle un offset, los arrays son muy eficientes en términos de acceso a memoria. Es $O(1)$ acceder a cualquier elemento de un array.

1.4.2.6 Tipo referencia (C/C++)

En C++, se puede utilizar el tipo `&` para definir una referencia a una variable. Una referencia es similar a un puntero, pero más seguro y más fácil de usar. Una referencia no puede ser nula y no puede ser reasignada. Una referencia es simplemente un alias para una variable.

```
int x = 10;
int& ref = x;

cout << x; // Imprime 10
cout << ref; // Imprime 10
```

Son muy útiles para pasar argumentos a funciones por referencia.

1.4.2.7 Paso de parámetros por valor vs por referencia

En C/C++, los parámetros de una función pueden pasarse por valor o por referencia. Es decir, una función puede recibir una copia del valor de una variable o la dirección de memoria de la variable.

- **Por valor:** Se pasa una copia del valor de la variable. Los cambios a la variable dentro de la función no afectan a la variable original.

```
void foo(int x) {
    x = 20;
}

int main() {
    int x = 10;
    foo(x);
    cout << x; // Imprime 10
}
```

- **Por referencia:** Se pasa la dirección de memoria de la variable. Los cambios a la variable dentro de la función afectan a la variable original.

```
void foo(int& x) {
    x = 20;
}
```

```
int main() {
    int x = 10;
    foo(x);
    cout << x; // Imprime 20
}
```

o bien, en C:

```
void foo(int* x) {
    *x = 20;
}

int main() {
    int x = 10;
    foo(&x);
    cout << x; // Imprime 20
}
```

1.4.2.8 Buenas prácticas al usar punteros

- **Inicializar punteros:** Siempre inicializa los punteros. Un puntero no inicializado puede contener cualquier valor (random value) y puede causar errores difíciles de depurar o causar un segmentation fault al de-referenciarlo.

```
int* ptr = NULL; // Correcto
int* ptr; // Incorrecto
```

- **Verificar punteros antes de usarlos:** Antes de usar un puntero, verifica que no sea NULL.

```
if (ptr != NULL) {
    // Usar el puntero
}
```

- **Liberar memoria:** Siempre libera la memoria asignada dinámicamente cuando ya no la necesites para evitar fugas de memoria.

- **Evitar punteros colgantes (dangling pointers) :** Después de liberar memoria, establece el puntero a NULL para evitar el uso accidental de punteros colgantes.

```
free(ptr);
ptr = NULL;
```

- **Usar const cuando sea posible:** Si un puntero no debe modificar los datos a los que apunta, decláralo como `const`.

```
const int* ptr = &x;
```

- **Evitar aritmética de punteros compleja:** La aritmética de punteros puede ser propensa a errores. Evítala si es posible o úsala con cuidado.

```
int arr[10];
int* ptr = arr;
ptr += 2; // Apunta al tercer elemento del array
```

- **No retornar desde una función, un puntero a una variable del stack:** Si retornas un puntero a una variable del stack, la variable se libera cuando la función termina y el puntero se convierte en un puntero colgante.

```
int* foo() {
    int temp = 50;
    return &temp;
}

void bar() {
    int temp = 66;
    return;
}

int main() {
    int* r = foo();
    cout << *r; // Imprime 50
    bar();
    cout << *r; // Imprime 60
}
```

1.4.3 Punteros en lenguajes manejados

En lenguajes manejados como Java, C# y Python, los punteros no son accesibles directamente. En su lugar, se utilizan referencias. Una referencia es similar a un puntero, pero más seguro y más fácil de usar. Una referencia no puede ser nula y no puede ser reasignada. Una referencia es simplemente un alias para un objeto.

En Java, por ejemplo, se utilizan referencias para acceder a objetos. Por ejemplo:

```
class MyClass {
    int x;
}

public class Main {
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.x = 10;

        MyClass obj2 = obj; // Shallow-Copy
        System.out.println(obj.x); // Imprime 10
    }
}
```

La mayoría de los lenguajes manejados tienen recolección de basura, lo que

significa que no es necesario liberar la memoria manualmente. El recolector de basura se encarga de liberar la memoria automáticamente cuando un objeto ya no es accesible. Utiliza un algoritmo de marcado y barrido para determinar qué objetos son accesibles y cuáles no. Los objetos inaccesibles se marcan para ser liberados. Para determinar si un objeto es accesible, el recolector de basura sigue las referencias a través de los objetos.

Otros lenguajes modernos como Rust, tienen un sistema de tipos que garantiza la seguridad de la memoria sin necesidad de un recolector de basura. Rust utiliza un sistema de tipos basado en el concepto de *ownership* y *borrowing* para garantizar la seguridad de la memoria. Por ejemplo:

```
fn main() {  
    let mut x = 10;  
    let y = &x;  
    let z = &x;  
    println!("{}", x); // Imprime 10  
}
```

El código anterior no compilará en Rust. Rust garantiza que no haya referencias múltiples a un objeto mutable. En este caso, `x` es mutable, pero `y` y `z` son referencias inmutables. Rust garantiza que no haya referencias múltiples a un objeto mutable para evitar condiciones de carrera y errores de memoria.

1.5 Referencias adicionales

- Modern Operating Systems, Andrew S. Tanenbaum, Herbert Bos, Pearson, 2014.
- <https://www.geeksforgeeks.org/memory-layout-of-c-program/>
- <https://www.geeksforgeeks.org/cpp-pointer-operators/>

Chapter 2

Análisis de algoritmos

En este capítulo se aborda el tema de análisis de algoritmos. Dicho conocimiento es esencial para el diseño de algoritmos eficientes y para la toma de decisiones en el desarrollo de software. La industria ha hecho estandar las entrevistas enfocadas en algoritmos y estructuras de datos junto con análisis de complejidad.

2.1 Definición de Algoritmo

Es un procedimiento para cumplir una tarea. Es la idea detrás de un programa.

¿Programa vs Algoritmo?

Un programa es la implementación de un algoritmo en un lenguaje de programación.

Opera sobre un problema bien definido, toma un input y lo transforma en un output deseado. Un ejemplo cotidiano puede ser una receta de cocina:

- Input: Ingredientes
- Output: Comida
- Instrucciones: Algoritmo

Las características de un algoritmo son:

- No son ambiguos: cada paso tiene un solo significado
- Entrada bien definida: Debe ser clara y consistente
- Salida bien definida: Debe indicar que tipo de output genera
- Finitos: Terminan en un tiempo determinado.
- Factibles: Pueden ser ejecutados con los recursos tecnológicos disponibles

Un algoritmo puede ser representado de varias formas:

- Pseudo código
- Lenguaje natural

- Definición formal
- Diagramas de flujo

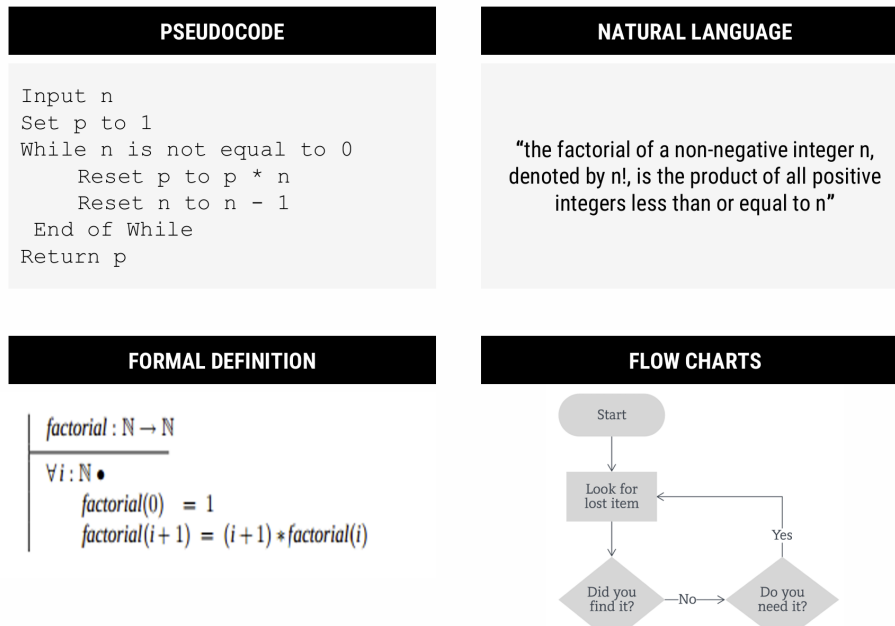


Figure 2.1: Representación de un algoritmo

2.2 Análisis de Algoritmos

En términos simples, analizar un algoritmo es el proceso para determinar la eficiencia de un algoritmo, medida en términos de tiempo y espacio. Esto no es una práctica de tiempos recientes, sino que se remonta a los inicios de la computación. Por ejemplo, Charles Babbage observó que:

“As soon as an Analytic Engine exists, it will necessarily guide the future course of the science. Whenever any result is sought by its aid, the question will arise—By what course of calculation can these results be arrived at by the machine in the **shortest** time?”

De manera similar y décadas después, Turing observó:

“It is convenient to have a measure of the amount of work involved in a computing process, even though it be a very crude one. We may count up the number of times that various elementary operations are applied in the whole process”

Ambos visionarios se referían a la necesidad de determinar la eficiencia de un algoritmo, con el fin de buscar la optimización del mismo o de comparar entre otros algoritmos de la misma “familia”.

La realidad es que hay muchos factores que pueden afectar la eficiencia de un algoritmo. Desde aspectos tecnológicos como el hardware, el sistema operativo, el lenguaje de programación, hasta aspectos más abstractos como la complejidad del algoritmo, la cantidad de datos, la pericia del programador, entre otros. Sea cual sea el factor, se desea analizar algoritmos para:

- Predecir comportamiento de un algoritmo y el programa que lo implementa.
- Comparar distintos algoritmos para el mismo propósito.
- Conociendo el comportamiento, podemos optimizar
- Clasificar el algoritmo según complejidad.

En las siguientes secciones se abordan dos formas de analizar algoritmos: *análisis empírico* y *análisis teórico*.

2.3 Análisis empírico de algoritmos

El término empírico se refiere a algo que se basa en la experiencia y la observación. En el análisis empírico, se evalúa el desempeño de un algoritmo ejecutándolo, conocido como *benchmarking*. El proceso entonces sería:

1. Marcar un tiempo de inicio
2. Ejecutar un programa con un input dado
3. Marcar el tiempo final
4. Calcular el tiempo transcurrido
5. Ejecutar la prueba varias veces, calcular mediana

Aunque pueda parecer contra-intuitivo, el análisis empírico es muy utilizado en la práctica profesional dado a que hay ambientes controlados donde se puede medir el desempeño de un algoritmo.

Profiling es análisis empírico. Esta es una técnica de análisis dinámico para encontrar cuellos de botella o secciones del código de un programa con mal rendimiento. Provee una visión completa: tiempo de ejecución, uso de memoria, uso de CPU, etc.

El problema clave de análisis empírico es que depende de la máquina en la que se ejecuta el algoritmo. Por lo tanto, no es posible generalizar los resultados obtenidos. No es lo mismo ejecutar un algoritmo en una máquina de hace 10 años que en una moderna.

2.4 Análisis teórico de algoritmos

Si el análisis empírico no provee una solución a prueba del tiempo y plataforma, ¿cómo se puede analizar un algoritmo? La respuesta es el análisis teórico.

El análisis teórico asume un modelo computacional en el que:

- Cada operación simple (+, -, *, /, =, ==, etc) toma tiempo constante. Estas son las operaciones elementales a las que se refería Turing.
- Los *loops* y subrutinas/funciones/métodos/procedimientos no son operaciones simples, sino que son la composición de muchas operaciones simples.
- Cada acceso a memoria toma tiempo constante.

Este modelo es una abstracción de la realidad, pero es perfecto para determinar el comportamiento de un algoritmo. No nos interesan los nano-segundos que toma una operación, sino el comportamiento general del algoritmo, **cual es la tasa de crecimiento de la cantidad de operaciones realizadas conforme el tamaño de la entrada.**

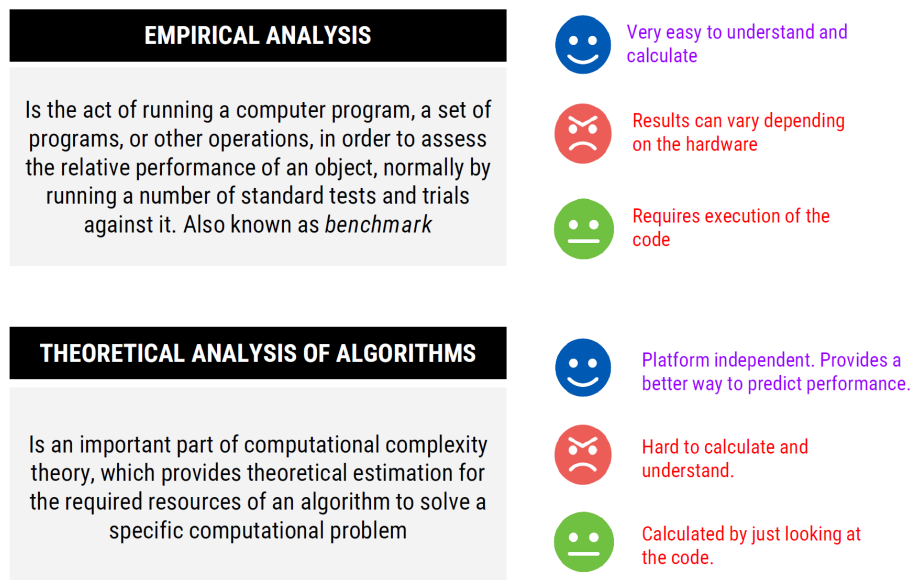


Figure 2.2: Comparación entre análisis empírico y teórico

Utilizando este modelo, podemos encontrar una función con base en el tamaño de la entrada. Por ejemplo, para el siguiente algoritmo

```
int max(int[] array) {
    int max = array[0];
    for (int i = 1; i < array.length; i++) {
```

```

        if (array[i] > max) {
            max = array[i];
        }
    }
    return max;
}

```

la función que describe el tiempo que se requiere para encontrar el máximo de un arreglo de tamaño n es (en el peor caso) sería

```

int max = array[0]; -----> 3T
for (int i = 1; i < array.length; i++) { ----> T + 2nT
    if (array[i] > max) { -----> 2T(n-1)
        max = array[i]; -----> 2T(n-1)
    }
}
return max; -----> T

f(n) = 6Tn + T

```

Encontrar la función a este nivel de detalle no es práctico. Imagínese el trabajo que implicaría una función como $f(n) = 12754n^2 + 4353n + 834\lg_2 n + 13546$. Más adelante veremos un enfoque que nos permite simplificar el trabajo.

2.4.1 Mejor, peor y caso promedio

Cuando se analiza un algoritmo, el **mejor caso** es el escenario en el que se realizan la menor cantidad de operaciones. Por ejemplo, en el algoritmo de búsqueda secuencial, el mejor caso es cuando el elemento buscado es el primer elemento del arreglo.

Al enfocarse en el **peor caso**, nos enfocamos en el escenario que causa que la mayor cantidad de operaciones se ejecute. Por ejemplo, en el algoritmo de búsqueda secuencial, el peor caso es cuando el elemento buscado está al final del arreglo.

En el **caso promedio**, tomamos todos los posibles inputs y calculamos el tiempo promedio que tomaría el algoritmo. Se suman todos los valores calculados y se divide entre el total de entradas. Por ejemplo, para el algoritmo de búsqueda secuencial, el caso promedio es cuando el elemento buscado está en cualquier posición del arreglo.

- Si el elemento está en la posición i se ejecutan i operaciones.
- Promedio de comparaciones es: $(1 + 2 + 3 + \dots + n) / n = n(n+1)/2$
- Promedio por elemento: $n(n+1)/2/n = (n+1)/2$

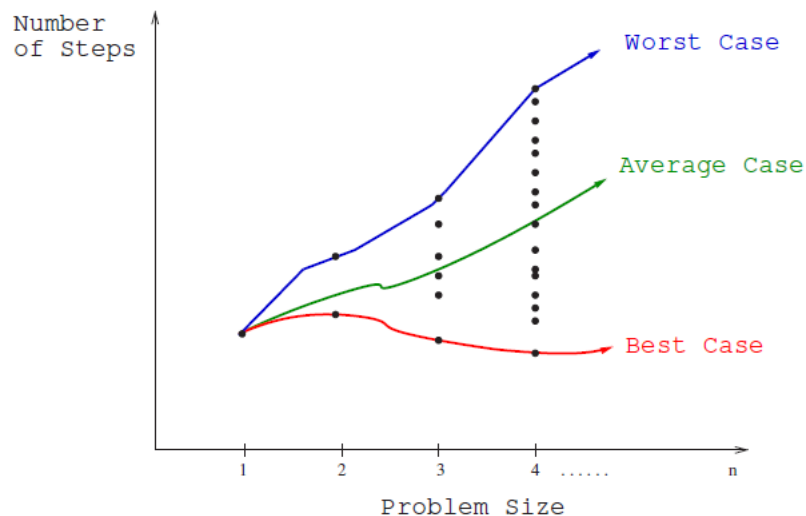


Figure 2.3: Comparativa de mejor, peor y caso promedio

El **peor caso** es el que resulta más útil para el análisis de algoritmos. Es el que nos da una idea de la eficiencia del algoritmo en el peor escenario posible. Conociendo lo peor que puede llegar, y siendo este aceptable, podemos asumir que el algoritmo es eficiente.

2.4.2 Análisis asintótico y notaciones comunes

Suponga que usted necesita enviar un archivo a un amigo en Guanacaste. ¿Qué es más rápido, enviarlo por correo/FTP o llevarlo personalmente? Asumiendo que ir a Guanacaste sin presas, tarda siempre 3 horas, podríamos tener el siguiente gráfico:

No importa que tan grande sea el archivo, llevarlo físicamente siempre tarda lo mismo. Por medio electrónico, el tiempo de transferencia depende del tamaño del archivo ($O(n)$) y en algún momento será mayor que las 3 horas que tarda llevarlo físicamente ($O(3)$).

Análisis asintótico busca encontrar la función que represente el crecimiento con respecto a n , sin entrar en detalles abrumadores de la función, es decir, podemos descartar los términos de menor relevancia.

Considere la función que calculamos tiempo atrás: $f(n) = 4T + 6nT$. Enfocándose en *análisis asintótico*, se descartan los términos de menor relevancia, es decir, los que no son significativos para el crecimiento de la función. De igual forma las constantes se eliminan. Por lo tanto, dicha función se puede expresar como

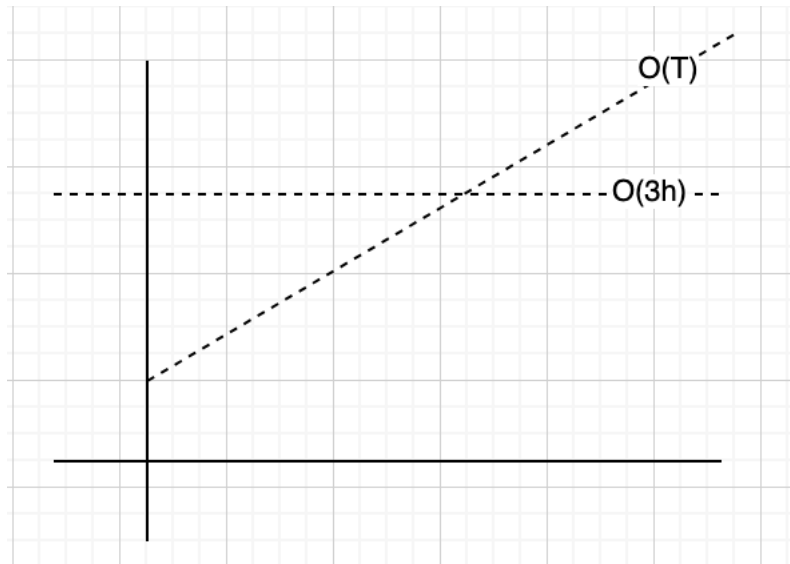


Figure 2.4: Ejemplo cotidiano sobre análisis asintótico

$$f(n) = O(n)$$

Lo que nos interesa en análisis asintótico, es la escalabilidad del algoritmo.

$$O(n^2 + n) \Rightarrow O(n^2)$$

$$O(n + \log(n)) \Rightarrow O(n)$$

$$O(5 * 2^n + 100n^2) \Rightarrow O(2^n)$$

Por ejemplo, considere el siguiente algoritmo:

```
for (int i = 0; i < vector.length(); i++) {
    foo(vector[i]); //Suponga que foo es tiempo constante
}
```

La gráfica de este algoritmo sería:

En caso que la constante cambie, se la función seguirá siendo lineal. Aunque la pendiente cambien, la tendencia del algoritmo seguirá siendo la misma.

2.4.3 Notación Big O, Big Omega y Big Theta

Son notaciones para describir la ejecución de un algoritmo en términos de su comportamiento asintótico.

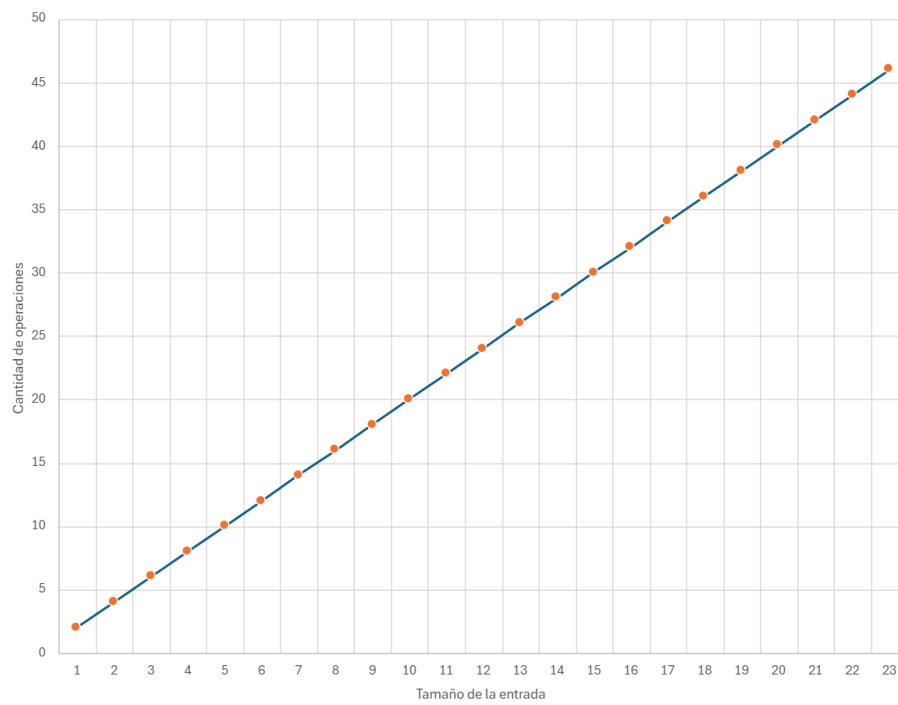


Figure 2.5: Gráfica de complejidad lineal

2.4.3.1 Big O

Describe el límite superior. Por ejemplo $O(n^2)$, $O(n)$, $O(2^n)$. El algoritmo es al menos tan rápido como este límite. No sobrepasa el límite dictado por Big O. Se define formalmente como $f(n) = O(g(n))$, donde $c * g(n)$ es un límite superior para $f(n)$. Es decir, existe una constante c tal que $f(n) \leq c * g(n)$ para todo n mayor que un n dado.

$f(n)$ es la función que representa el algoritmo con precisión, por ejemplo, $f(n) = 3n^2 - 100n + 6$. $g(n)$ es el intento de clasificar nuestra función, por ejemplo, $g(n) = n^2$. La constante que multiplica a $g(n)$ la podemos escoger arbitrariamente, por ejemplo, $c = 3$. Si graficamos $g(n) = 3n^2$ y $f(n)$, veremos que $3n^2$ es siempre mayor que $f(n)$, concluyendo que $f(n) = O(n^2)$.

2.4.3.2 Big Omega

Describe el límite inferior. Es decir, el algoritmo tendrá un comportamiento al menos tan lento como este límite. Se define formalmente como $f(n) = \Omega(g(n))$, donde $c * g(n)$ es un límite inferior para $f(n)$. Es decir, existe una constante c tal que $f(n) \geq c * g(n)$ para todo n mayor que un n dado.

Por ejemplo, dada la función $f(n) = 3n^2 - 100n + 6 = \Omega(n^2)$, porque para $c = 2$, $2n^2 < f(n)$ cuando $n > 100$.

2.4.3.3 Big Theta

Describe el comportamiento exacto del algoritmo. Es decir, el algoritmo se comporta exactamente como esta función. Es el límite ajustado, *Big O* y *Big Omega*. Se define formalmente como $f(n) = \Theta(g(n))$, donde $c1 * g(n)$ es un límite inferior y $c2 * g(n)$ es un límite superior para $f(n)$. Es decir, existen constantes $c1$ y $c2$ tal que $c1 * g(n) \leq f(n) \leq c2 * g(n)$ para todo n mayor que un n dado.

Por ejemplo, dada la función $f(n) = 3n^2 - 100n + 6 = \Theta(n^2)$, porque $O(n^2)$ y $\Omega(n^2)$ aplican..

El mejor, peor y caso promedio, se puede describir con cualquiera de las notaciones de complejidad asintótica. Es incorrecto que *Big O* solo se refiere al peor caso, *Big Omega* es el mejor y *Big Theta* es el promedio.

2.4.3.4 Sobre la complejidad espacial

Aunque lo visto hasta el momento se enfoca en complejidad en tiempo, la cantidad de memoria que el algoritmo requiere se puede describir con Big-O, Big-Omega y Big-Theta también.

```
int sum(int n) {
    if (n <= 0) {
        return 0;
    }
}
```

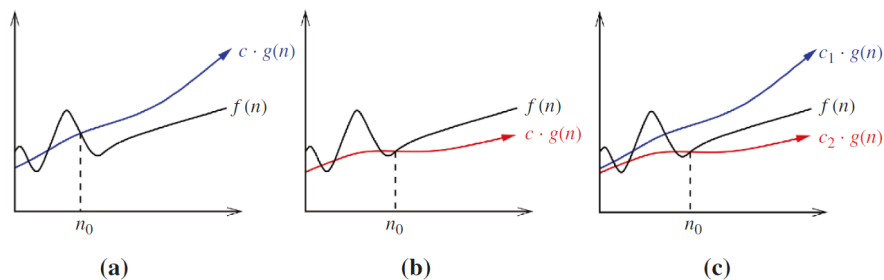


Figure 2.6: Visualización de Big O, Big Omega y Big Theta

```

    }
    return n + sum(n-1);
}

```

La complejidad espacial de este algoritmo es $O(n)$ dado que se requiere almacenar n llamadas recursivas en la pila de llamadas.

¿Cuál es la complejidad espacial de este algoritmo?

```

int foo(int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += bar(i);
    }
    return sum;
}

int bar(int a, int b) {
    return a + b;
}

```

No hay llamadas anidadas $\Rightarrow O(1)$

2.4.3.5 Big-O conocidas

El siguiente gráfico muestra las complejidades comunes con las que se clasifican muchos de los algoritmos conocidos:

2.5 Determinar la complejidad de un algoritmo

Para determinar la complejidad de un algoritmo, dependerá del tipo de instrucciones que se utilicen en el código.

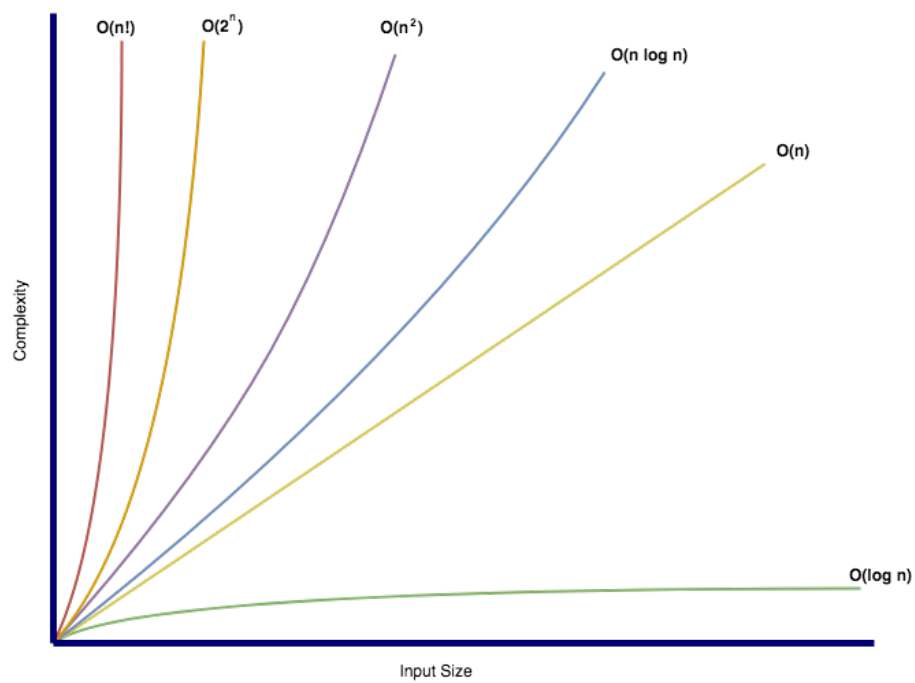


Figure 2.7: Gráfico de Big-O conocidas

2.5.1 Secuencia de instrucciones

Para código de la forma:

```
statement 1;  
statement 2;  
...  
statement n;
```

El total se calcula sumando la complejidad de cada instrucción:

```
complejidad(statement 1) + complejidad(statement 2) + ... +  
complejidad(statement n)
```

2.5.2 If-Then-Else

Para código de la forma:

```
if (condition) {  
    statement 1;  
} else {  
    statement 2;  
}
```

La complejidad es $\max(\text{complejidad}(\text{statement 1}), \text{complejidad}(\text{statement 2}))$

2.5.3 Loops

Para código de la forma:

```
for (int i = 0; i < n; i++) {  
    statement;  
}
```

La complejidad es $n * \text{complejidad}(\text{statement})$. Asumiendo que la complejidad de `statement` es $O(1)$, la complejidad del loop es $O(n)$.

Ahora considere el siguiente código con loops secuenciales:

```
for (int i = 0; i < n; i++) {  
    statement 1;  
}  
for (int j = 0; j < m; j++) {  
    statement 2;  
}
```

La complejidad es $O(n + m)$.

2.5.4 Anidamiento

Para código de la forma:

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        statement;
    }
}

```

La complejidad es $n * m * \text{complejidad}(\text{statement})$. Si ambos loops son de tamaño n , la complejidad es $O(n^2)$.

Conforme se anidan más loops, la complejidad crece exponencialmente. Por ejemplo, si se anidan 3 loops, la complejidad es $O(n^3)$.

2.5.5 Complejidad logarítmica

Para código de la forma:

```

int i = n;
while (i > 0) {
    statement;
    i = i / 2;
}

```

La complejidad es $O(\log(n))$. En cada iteración, i se divide por 2. ¿Cómo se llega a esta conclusión?

- En la primera iteración, i es n
- En la segunda iteración, i es $n/2$
- En la tercera iteración, i es $n/4$

En general, i es $n/2^k$ en la k -ésima iteración. La complejidad es $O(\log_2(n))$ porque k es el número de veces que se puede dividir n por 2 hasta llegar a 1.

2.5.6 Recursión

Para la recursión no hay una regla general, dado que depende de lo que haga el código. Por ejemplo, para el siguiente código:

```

int foo(int n) {
    if (n <= 0) {
        return 0;
    }
    return n + foo(n-1);
}

```

La complejidad es $O(n)$. La función se llama n veces, decrementando n en cada llamada.

Para el siguiente código:

```

int foo(int n) {
    if (n <= 0) {

```

```

    return 0;
}
return n + foo(n-1) + foo(n-1);
}

```

La complejidad es $O(2^n)$. En cada llamada, se hacen dos llamadas recursivas. ¿Cómo se llega a esta conclusión?

Gráficamente, se puede ver que la cantidad de llamadas se duplica en cada nivel de la recursión, como se muestra en la siguiente figura:

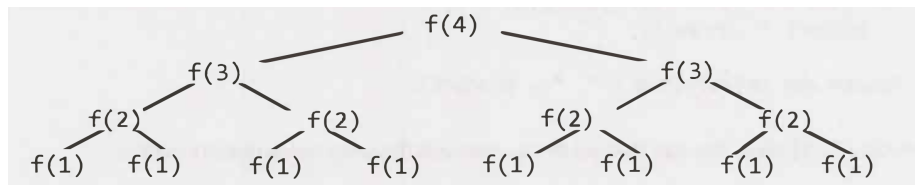


Figure 2.8: Visualización de recursión

Tabulando los datos anteriores:

Nivel	# de Nodos	
0	1	2^0
1	2	2^1
2	4	2^2
3	8	2^3
4	16	2^4

Por lo tanto, hay $2^{(n+1)} - 1$ nodos y la complejidad es $O(2^n)$. En muchos casos, la complejidad recursiva se puede generalizar como $O(\text{ramas}^{\text{profundidad}})$, donde *branches* es el número de llamadas recursivas y *depth* es la profundidad de la recursión.

2.5.7 Más ejemplos

2.5.7.1 Ejemplo #1

Para el siguiente código:

```

void foo(int[] array) {
    int sum = 0;
    int product = 1;
    for (inti= 0; i < array.length; i++) {
        sum += array[i];
    }
    for (int i= 0; i < array.length; i++) {

```



```

        product*= array[i];
    }
    System.out.println(sum + ", " + product)
}

```

La complejidad sería $O(n)$. Iterar dos veces el arreglo no importa puesto que sería $O(2n) = O(n)$.

2.5.7.2 Ejemplo #2

Para el siguiente código:

```

void printUnorderedPairs(int[] array) {
    for (int i= 0; i < array.length; i++) {
        for (int j = i + 1; j < array.length; j++) {
            System.out.println(array[i] + "," + array[j]);
        }
    }
}

```

Se realizan $(N-1) + (N-2) + \dots + 1 = N(N-1)/2$ operaciones. La complejidad es $O(n^2)$.

2.5.7.3 Ejemplo #3

El siguiente código:

```

void printUnorderedPairs(int[] arrayA, int[] arrayB) {
    for (inti= 0; i < arrayA.length; i++) {
        for (int j = 0; j < arrayB.length; j++) {
            if (arrayA[i] < arrayB[j]) {
                System.out.println(arrayA[i] + "," + arrayB[j]);
            }
        }
    }
}

```

Tiene una complejidad de $O(ab)$, donde a es el tamaño de *arrayA* y b es el tamaño de *arrayB*.

2.5.7.4 Ejemplo #4

El siguiente código:

```

void printUnorderedPairs(int[] arrayA, int[] arrayB) {
    for (int i= 0; i < arrayA.length; i++) {
        for (int j = 0; j < arrayB.length; j++) {
            for (int k = 0; k < 100000; k++) {
                System.out.println(arrayA[i] + "," + arrayB[j]);
            }
        }
    }
}

```

```

    }
  }
}

```

Tiene una complejidad de $O(ab)$, donde a es el tamaño de *arrayA* y b es el tamaño de *arrayB*. El loop interno no afecta la complejidad dado que es $O(100000)$.

2.5.7.5 Ejemplo #4

```

void reverse(int[] array) {
    for (int i= 0; i <array.length/ 2; i++) {
        int other= array.length - i - 1;
        int temp= array[i];
        array[i] = array[other];
        array[other] = temp;
    }
}

```

En este caso, se recorre solo la mitad del arreglo. La complejidad es $O(n/2) = O(n)$.

2.5.7.6 Ejemplo #5

Para sumar todos los nodos de un BST:

```

int sum(Node node) {
    if (node == null) {
        return 0;
    }
    return sum(node.left) + node.value + sum(node.right);
}

```

Dado que se recorren todos los nodos del árbol, la complejidad es $O(n)$.

2.5.7.7 Ejemplo #6

```

boolean isPrime(int n) {
    for (int x = 2; x <= sqrt(n); x++) {
        if (n % x == 0) {
            return false;
        }
    }
    return true;
}

```

La complejidad es $O(\sqrt{n})$.

2.5.7.8 Ejemplo #7

```
int fib(int n) {
    if (n <= 0) return 0;
    else if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);
}
```

La complejidad es $O(2^n)$.

2.5.7.9 Ejemplo #8

```
int factorial(int n) {
    if (n < 0) {
        return -1;
    } else if (n == 0) {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}
```

La complejidad es $O(n)$. Se recorren n llamadas recursivas.

2.6 Revisitando los algoritmos y estructuras de datos

El peor caso de algunas estructuras de datos comunes se resume en la siguiente tabla:

2.7 Referencias

- Skiena S. 2020. The Algorithm Design Manual. Springer.
- <https://www.geeksforgeeks.org/what-is-algorithm-and-why-analysis-of-it-is-important/>
- Laakmann Gayle L. 2020. Cracking the Coding Interview. 6th Edition. CareerCup.

Data structure	Access	Search	Insertion	Deletion
Array	$O(1)$	$O(N)$	$O(N)$	$O(N)$
Stack	$O(N)$	$O(N)$	$O(1)$	$O(1)$
Queue	$O(N)$	$O(N)$	$O(1)$	$O(1)$
Singly Linked list	$O(N)$	$O(N)$	$O(N)$	$O(N)$
Doubly Linked List	$O(N)$	$O(N)$	$O(1)$	$O(1)$
Hash Table	$O(N)$	$O(N)$	$O(N)$	$O(N)$
Binary Search Tree	$O(N)$	$O(N)$	$O(N)$	$O(N)$
AVL Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$
Binary Tree	$O(N)$	$O(N)$	$O(N)$	$O(N)$
Red Black Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$

Figure 2.9: Resumen de complejidad Big O

Chapter 3

Diseño de Algoritmos

En este capítulo se exploran algunos paradigmas de diseño de algoritmos utilizados en muchos de los algoritmos conocidos y utilizados en la actualidad. Analizar estos paradigmas, permitirá utilizar técnicas similares para nuevos problemas de ingeniería de software.

3.1 Divide y conquista

Divide y conquista es uno de los paradigmas introducidos en muchos de los cursos de programación introductorios. En términos generales, se divide el problema recursivamente en sub-problemas más pequeños, se resuelven de forma recursiva y se combinan las soluciones para obtener la solución final. Cuando la combinación toma menos tiempo que la resolución de los sub-problemas, se obtiene una mejora en la eficiencia.

La **fase de división** incluye:

- Divide el problema en sub-problemas más pequeños
- Los sub-problemas deben ser de la misma forma que el problema original, pero más pequeños
- Se divide hasta llegar al caso base

La **fase de conquista** incluye:

- Resolver cada sub-problema individualmente
- Si el sub-problema es lo suficientemente pequeño, se resuelve de forma directa
- El objetivo es resolver los sub-problemas independientemente

La **fase de combinación** incluye:

- Combinar las soluciones de los sub-problemas para obtener la solución del problema original

- Una vez que los sub-problemas se resuelven, se combinan para obtener la solución final
- El objetivo es obtener la solución del problema original

3.1.1 Ejemplos de algoritmos de divide y conquista

3.1.1.1 Merge Sort

Merge Sort es un algoritmo de ordenamiento que utiliza el paradigma de divide y conquista. La idea es dividir el arreglo en dos mitades, ordenar cada mitad y luego combinar las dos mitades ordenadas.

Visualmente, el algoritmo se ve de la siguiente manera:

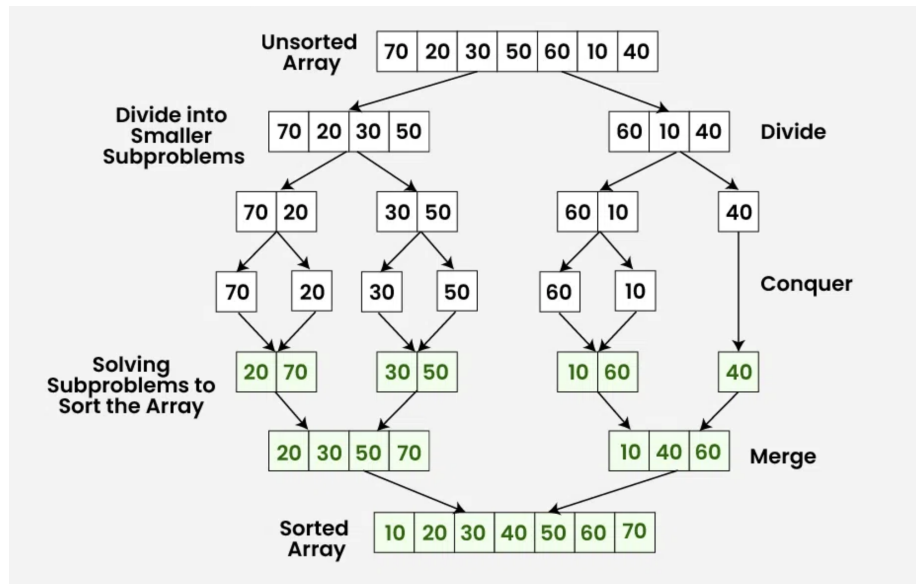


Figure 3.1: Visualización de MergeSort

En este algoritmo, en la **fase de división**, se generan $\log_2(n)$ niveles. En la **fase de merge**, combinar cada nivel toma $O(n)$ tiempo. Por lo tanto, la complejidad de tiempo de Merge Sort es $O(n \log n)$.

3.1.1.2 Búsqueda binaria

Búsqueda binario, tal y como lo hemos visto en cursos anteriores, es un algoritmo de búsqueda que utiliza el paradigma de divide y conquista. La idea es dividir el arreglo en dos mitades, comparar el elemento con el valor medio y decidir en qué mitad continuar la búsqueda.

```
int binary_search(item_type s[], item_type key, int low, int high) {
    int middle;
```

```

    if (low > high) {
        return (-1);
    }
    middle = (low + high) / 2;
    if (s[middle] == key) {
        return(middle);
    }
    if (s[middle] > key) {
        return(binary_search(s, key, low, middle-1));
    } else {
        return(binary_search(s, key, middle + 1, high));
    }
}

```

La complejidad de tiempo de la búsqueda binaria es $O(\log n)$. En cada paso, el tamaño del problema se reduce a la mitad y no hay necesidad de mezclar los resultados.

3.1.1.3 Encontrar el máximo de un arreglo

Para encontrar el máximo de un arreglo, la solución trivial sería recorrer el arreglo y comparar cada elemento con el máximo actual. La complejidad de tiempo de esta solución es $O(n)$:

```

int findMax(int[] a, int n)
{
    int max = Integer.MIN_VALUE;
    for (int i = 0; i < n; i++)
    {
        if (a[i] > max)
        {
            max = a[i];
        }
    }
    return max;
}

```

Utilizando divide y conquista, se puede resolver de la siguiente manera:

```

static int findMax(int[] a, int lo, int hi)
{
    if (lo > hi)
    {
        return Integer.MIN_VALUE;
    }
    if (lo == hi)
    {
        return a[lo];
    }
}

```

```

    }
    int mid = (lo + hi) / 2;
    int leftMax = findMax(a, lo, mid);
    int rightMax = findMax(a, mid + 1, hi);
    return Math.max(leftMax, rightMax);
}

```

La complejidad de tiempo de este algoritmo es $O(n)$. El análisis refleja que aunque sea $O(n)$, realiza menos comparaciones que la solución trivial.

Investigar: ¿Por qué la complejidad de tiempo de este algoritmo es $O(n)$?
¿Por qué no es $O(\log n)$?

3.1.1.4 Paralelismo en divide y conquista

Computación paralela es una técnica que permite realizar múltiples tareas simultáneamente aprovechando múltiples núcleos de procesamiento. Cada partición del problema se puede asignar a un núcleo de procesamiento, lo que permite reducir el tiempo de ejecución.

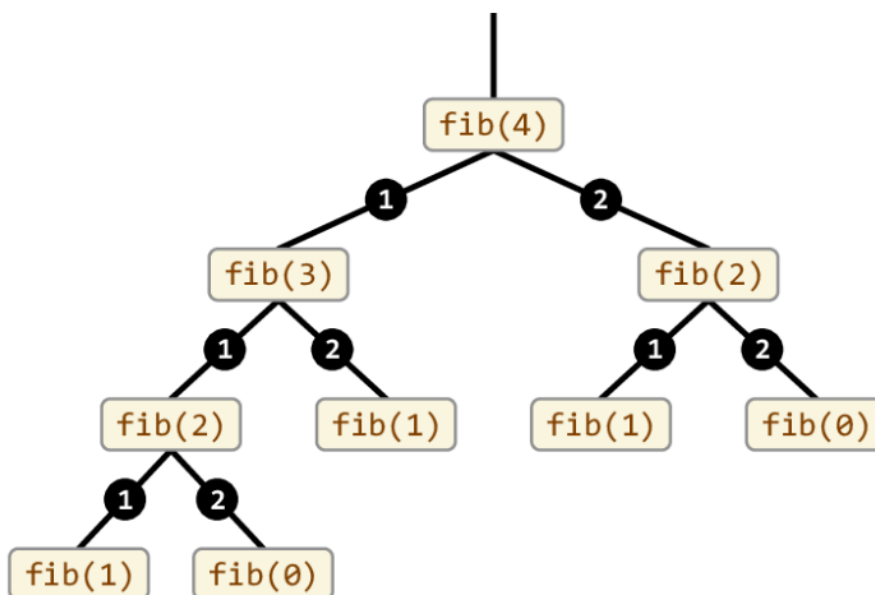
Investigar: ¿Cómo se puede implementar paralelismo en el algoritmo de búsqueda binaria y en *merge sort*?

3.2 Programación dinámica (DP)

Aunque tiene el término *programación* en su nombre, no se refiere a escritura de código fuente. Acuñado por Richard Bellman en los años 50, *programar* se refiere a *planificar*, es decir, planificar óptimamente procesos de múltiples etapas. - Comparte similitudes con la técnica de *divide y vencerás*. - DP divide problemas en sub-problemas y *memoiza* las soluciones de los sub-problemas para resolverlos una *sola vez*. - En DP, los sub-problemas se traslapan, es decir, el resultado de uno puede ayudar a resolver otro.

Memoización vs Memorización Memoización se refiere a una técnica para optimizar recordando. Memorizar se refiere a poner algo en la memoria.

Por ejemplo, para calcular *fibonacci(4)* utilizando un enfoque tradicional:



Se puede notar un claro traslape de sub-problemas, lo que implica un desperdicio de recursos computacionales. Utilizando un enfoque de DP, el problema se puede resolver de la siguiente manera:

```

int fib(n) {
    mem[n + 2]; // -----> En caso que N sea 0 o 1
    mem[0] = 0;
    mem[1] = 1;

    for (i = 2; i < n + 1; i++) {
        mem[i] = mem[i - 1] + mem[i - 2];
    }
    return mem[n];
}

```

El código anterior utilizaría internamente, un arreglo que se vería de la siguiente manera:

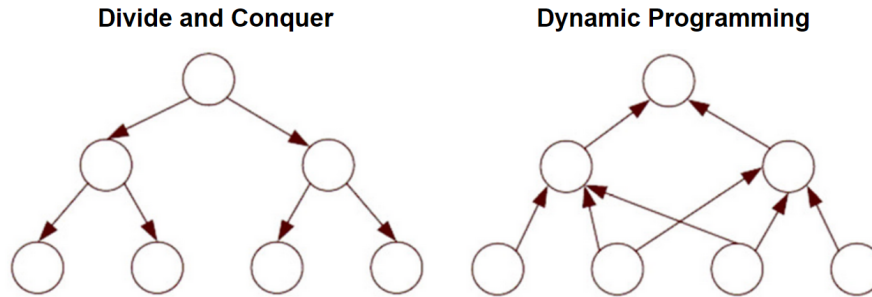
```

mem[0] = 0
mem[1] = 1
mem[2] = 1
mem[3] = 2
mem[4] = 3

```

En el enfoque divide y conquista, el algoritmo de Fibonacci tiene una complejidad de tiempo de $O(2^n)$. En cambio, con DP, la complejidad de tiempo se reduce a $O(n)$.

El siguiente diagrama ilustra el enfoque DP vs Divide y Vencerás de forma general:



3.2.1 Ejemplo de programación dinámica: Longest Common Subsequence (LCS)

Cadena más larga común entre dos strings, no necesariamente contigua. Por ejemplo,

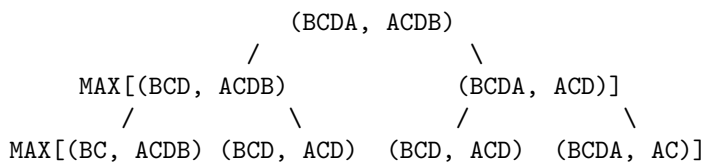
S1 = "BCDAACD"
S2 = "ACDBAC"

Tiene las cadenas comunes: BC, CDAC, DAC, ...

Implementándolo usando el enforque tradicional:"

```
// Returns length of LCS for X[0..m-1], Y[0..n-1]
int lcs(String X, String Y, int m, int n)
{
    if (m == 0 || n == 0)
        return 0;
    if (X.charAt(m - 1) == Y.charAt(n - 1))
        return 1 + lcs(X, Y, m - 1, n - 1);
    else
        return max(lcs(X, Y, m, n - 1),
                    lcs(X, Y, m - 1, n));
}
```

Para las cadenas BCDA y ACDB, se generaría un árbol de recursión de la siguiente manera:



Claramente vemos como se resolvería varias veces el mismo sub-problema, siendo $O(2^{nm})$. Utilizando DP, se puede resolver de la siguiente manera:

Se crea una matriz de tamaño $(m+1) \times (n+1)$ y se llena de la siguiente manera:

```
static int lcs(String X, String Y, int m, int n)
{
    int[, ] L = new int[m + 1, n + 1];

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0)
                L[i, j] = 0;
            else if (X[i - 1] == Y[j - 1])
                L[i, j] = L[i - 1, j - 1] + 1;
            else
                L[i, j] = max(L[i - 1, j], L[i, j - 1]);
        }
    }
    return L[m, n];
}
```

	A	C	D	B
0	0	0	0	0
B	0	0	0	1
C	0	0	1	1
D	0	0	1	2
A	0	1	1	2

Esto resulta en complejidad temporal $O(nm)$

3.3 Backtracking

- Popularizado por Henry Lehmer, matemático estadounidense.
- Es una forma metódica de probar distintas secuencias de decisiones hasta encontrar una que funcione
- Se puede conceptualizar como un árbol de decisiones
 - Cada nodo del árbol solo puede ver sus hijos directos
 - Si un nodo conduce a error, se regresa al anterior y se prueba con otro hijo

La estructura general en código se puede resumir como:

```
backtrack(x) {
    if (x == solucion) {
        return true;
    }
}
```

```

    }
    for (nodo in x.nodos) {
        if (nodo == solution) {
            return true;
        } else {
            return backtrack(nodo);
        }
    }
}
}

```

3.3.1 Ejemplo: el problema de las N-reinas

Dado un tablero de ajedrez de $N \times N$, colocar N reinas de tal forma que no se ataquen entre sí. Una reina puede atacar a otra si están en la misma fila, columna o diagonal.

```

bool solve(board[] [] col) {
    if (col == N) {
        return true;
    }
    for (int i = 0; i < N; i++) {
        if (isSafe(board, i, col)) {
            board[i][col] = 1;
            if (solve(board, col + 1)) {
                return true;
            }
            board[i][col] = 0;
        }
    }
    return false;
}
}

```

3.4 Algoritmos Probabilísticos

Son algoritmos que utilizan aleatoriedad para tener una mejora en desempeño sacrificando la confiabilidad de los resultados obtenidos:

- No se produce ningún resultado
- Se produce un resultado incorrecto
- Se produce una respuesta aproximada

Ejecuciones distintas pueden producir respuestas distintas.

3.4.1 Clasificación

- **Monte Carlo:** Algoritmos que **siempre** retornan un resultado, pero puede no ser correcto. Se intenta minimizar la probabilidad de error. Múltiples ejecuciones reducen dicha probabilidad.
- **Las Vegas:** Algoritmos que **siempre** retornan un resultado correcto, pero pueden producir ningún resultado. Múltiples ejecuciones reducen la probabilidad de no obtener un resultado.

3.4.2 Aleatoriedad

- Provisto por un generador de números random. Estos generadores son pseudo-aleatorios, ya que generan una secuencia de números que parecen ser aleatorios, pero son deterministas. El único valor random que existe en nuestra realidad es la decadencia radioactiva.
- Los generadores de pseudo-random generan números en una secuencia dentro de un rango y requieren un elemento inicial llamado semilla (seed). Cada número en la secuencia se genera a partir del anterior.
- Hay posibilidad de ciclos dado que un número puede repetirse en la secuencia.



3.4.2.1 Ejemplo de pseudo-random: Método de cuadrado medio

Eleve el número inicial (seed) al cuadrado y tome los dígitos del medio como el nuevo número. Por ejemplo, si el seed es 1234, el cuadrado es 1522756 y el número medio es 2275.

Una secuencia sería:

- 1234 (semilla)
- 2275 (1234^2)
- 5180 (2275^2)
- 6884 (5180^2)

y así sucesivamente. Dependiendo de la semilla, la secuencia puede ser cíclica muy rápido.

3.4.3 Algoritmos de Monte Carlo

Considere el siguiente requerimiento: “Dado un array de N elementos, determinar si hay un elemento que sea mayoritario, es decir que aparezca más de $N/2$ veces”

La solución trivial sería:

1. Recorra el array y cuente cuántas veces aparece cada elemento.
2. Si encuentra un elemento que aparece más de $N/2$ veces, retorne verdadero.

El problema es que la complejidad de este algoritmo es $O(N^2)$ y no es eficiente.

Utilizando un algoritmo de Monte Carlo, tendríamos:

```
boolean tieneElementoMayoritario(array, lenght) {
    i = random(0, n - 1);
    x = array[i];
    k = 0;
    for (j = 0; j < n; j++) {
        if (array[j] == x) {
            k++;
        }
    }
    return k > n / 2;
}
```

Como se puede notar, podría devolver **false** aún cuando no haya un elemento mayoritario. La probabilidad de error es de $1/2$.

Ejecutar varias veces el algoritmo reduce la probabilidad de error (si el psuedo-random es bueno). Después de k ejecuciones, la probabilidad de error es de $1/2^k$.

3.4.4 Algoritmos de Las Vegas

Características:

- No garantizan un resultado y no tiene un upper-bound en tiempo de ejecución. Siempre se obtiene un resultado correcto, pero no siempre se obtiene un resultado.
- Utilizados para explorar un espacio de soluciones de las cuales algunas son las correctas. Usan random para moverse en dicho espacio de soluciones. Si hay muchas soluciones correctas, la probabilidad de encontrar una es alta en poco tiempo.

Considere el problema de las n -reinas. La solución clásica es mediante backtracking. Sin embargo, un algoritmo de Las Vegas podría ser:

1. Colocar las reinas en posiciones aleatorias, una en cada columna
2. Verificar si hay colisiones
3. Si no hay colisiones, retornar la solución

4. Si hay colisiones, repetir el proceso

3.4.5 Estructuras de datos probabilísticas

- Proveen respuestas aproximadas a consultas sobre grandes sets de datos.
- Sacrifican precisión para fomentar eficiencia temporal.
- Utilizan *hashing* y *randomness* para reducir la complejidad espacial y temporal.

Una diferencia clave de las estructuras probabilísticas con respecto a los algoritmos probabilísticos, es que estos últimos no necesariamente se comportan de forma impredecible para el usuario. Es decir, pueden devolver resultados correctos. Las estructuras de datos probabilísticas, no dan respuestas definitivas.

3.4.5.1 Filtros de Bloom

Considere el siguiente escenario:

La página de registro de usuarios de un sitio web, permite al usuario especificar un *username*. No se permiten *username* duplicados. ¿Cómo se puede verificar si un *username* ya existe?

Algunas posibles soluciones serían:

- Aplicar búsqueda secuencial, pero el problema es que la complejidad es $O(N)$ y si tenemos millones de usuarios, la búsqueda sería muy lenta.
- Búsqueda binaria, que es mucho mejor, pero requeriría tener ordenados los usuarios y cargados en memoria.

Un filtro de Bloom permite determinar si un elemento pertenece a un set o no. Al ser probabilístico, puede dar falsos positivos (sí está), pero no falsos negativos.

- Utiliza un array de bits de tamaño fijo para representar todo el set de datos
- Agregar un elemento al set nunca falla, pero los falsos positivos aumentan conforme el set se llena
- Nunca genera falsos negativos
- No se pueden eliminar elementos del set

Implementación

Se utiliza un arreglo de bits de largo m .

Se necesitan k funciones de hash. Para agregar un elemento, se calculan las k funciones de hash y se setean los bits correspondientes a 1. Por ejemplo, si se define que se van a usar 3 funciones de hash, y se va a insertar la palabra “TEC” en el set:

$h_1(\text{TEC}) \% m = 1$ $h_2(\text{TEC}) \% m = 3$ $h_3(\text{TEC}) \% m = 5$

Ahora agregamos la palabra “QUIZ”:

$h1(\text{QUIZ}) \% m = 3$ $h2(\text{QUIZ}) \% m = 5$ $h3(\text{QUIZ}) \% m = 4$

Como se puede notar, en este caso, TEC y QUIZ tuvieron algunos resultados similares. El set se comienza a llenar, activando bits que no necesariamente corresponden a elementos diferentes.

Supongamos que se busca la palabra “PERRO”, la cual no se ha insertado aún, y que las funciones de hash generan los siguientes valores:

$h1(\text{PERRO}) \% m = 1$ $h2(\text{PERRO}) \% m = 4$ $h3(\text{PERRO}) \% m = 5$

Estos índices ya están en 1, por lo que el filtro de Bloom dirá que el elemento está en el set, cuando en realidad no lo está. Esto es un falso positivo.

Depende del tipo de aplicación, puede ser que esto sea aceptable. Por ejemplo, en el caso de la verificación de *username*, si el filtro de Bloom dice que el *username* ya existe, se puede hacer una verificación adicional para confirmar. Pero si el filtro dice que no existe, entonces no se necesita hacer nada más y se ahorra tiempo considerable.

Probabilidad de un falso positivo: $P(1 - [1 - 1/m]^{kn})^k$

3.4.5.1.1 Complejidad:

- Tiempo de inserción: $O(k)$
- Tiempo de búsqueda: $O(k)$
- Espacio requerido: $O(m)$

3.4.5.1.2 Selección de la función de hash

- Deben ser funciones rápidas
- Usar hash criptográfico proveerá una mayor estabilidad pero tiene un hit de performance muy importante

3.4.5.1.3 Aplicaciones conocidas de filtros de bloom

- Medium.com lo utiliza para identificar los post ya vistos por el usuario
- Cloudflare lo utiliza para identificar IPs maliciosas
- Google Chrome lo utiliza para identificar URLs maliciosas
- Apache Cassandra lo utiliza para buscar registros inexistentes en disco

3.5 Algoritmos Genéticos

Se fundamentan en el trabajo de John Holland en 1962, quien luego publica en 1975 su libro “Adaptation in Natural and Artificial Systems”. Los algoritmos genéticos son una técnica de optimización y búsqueda basada en la teoría de la evolución de Darwin.

En 1980 se aplican en problemas de optimización y en 1989 en problemas de aprendizaje automático.

Se pueden definir como una técnica de búsqueda para problemas de optimización y aprendizaje automático que simula el proceso de selección natural.

Se consideran como algoritmos heurísticos, ya que no garantizan la obtención de la solución óptima, pero sí una solución aceptable en un tiempo razonable.

Heurística significa “encontrar” en griego. Se refiere a la búsqueda de soluciones a problemas de forma rápida y eficiente, pero no necesariamente óptima. Es el pasado de *eureka*.

Utilizan técnicas inspiradas en la biología evolutiva, como la selección natural, la reproducción y la mutación.

3.5.1 Definiciones esenciales

- **Individuo:** Representa una solución al problema. Puede ser una cadena de bits, un vector de números, una estructura de datos, etc.
- **Población:** Conjunto de individuos/posibles soluciones.
- **Fitness:** Función que evalúa la calidad de una solución/individuos
- **Características:** Representan las propiedades de un individuo. Pueden ser los genes de un individuo.
- **Genoma:** Conjunto de características de un individuo.

Dependiendo del problema se pueden usar múltiples cromosomas para representar un individuo.

3.5.2 Representación de un individuo

El objetivo es optimizar el espacio de búsqueda escogiendo una representación adecuada para el problema. Usualmente se recomienda el uso de bit-vectors, donde cada entrada indica si cierta característica está presente o no.

3.5.3 Funcionamiento general

Inician con una población de individuos aleatorios. En cada generación, se evalúa el fitness de cada individuo, se seleccionan los mejores y se reproducen para generar una nueva generación.

La nueva población reemplaza a la anterior y se repite el proceso hasta que se cumple un criterio de parada.

¿Cuándo se detiene el algoritmo? Puede ser cuando se alcanza un número máximo de generaciones, cuando se alcanza un fitness mínimo, cuando se alcanza un fitness máximo, etc.

3.5.4 ¿Cómo seleccionar los padres?

Después de aplicar la función de fitness, se obtiene un grupo de posibles padres:

- Secuencia: 1 - 2, 3 - 4 . .
- Random

3.5.5 Introducir variabilidad

Después de seleccionar los padres, se aplica recombinación y mutaciones.

3.5.6 Recombinación

Se mezclan los genes de los padres para generar nuevos individuos. Suponiendo que los siguientes bit-vectors representan los padres:

	0 1 2 3 4 5
P1:	0 0 0 0 0 0
P2:	1 1 1 1 1 1
	~
	Cross-over point

Se generan los siguientes hijos:

	0 1 2 3 4 5
H1:	0 0 0 1 1 1
H2:	1 1 1 0 0 0

3.5.7 Mutación

Con base en alguna probabilidad determinada, se hace flip a algunos bits aleatorios de cada bit-vector de los hijos generados.

3.6 Referencias

- <https://www.geeksforgeeks.org/introduction-to-divide-and-conquer-algorithm/>

Chapter 4

Algoritmos de búsqueda

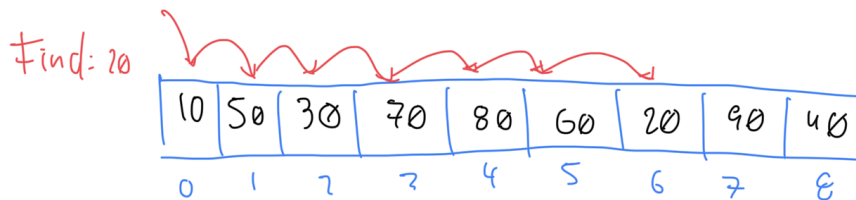
Este tema abarca los algoritmos de búsqueda en estructuras de datos como arrays y expande para incluir búsquedas en planos (aplicables a juegos por ejemplo). No incluye motores de búsqueda textuales.

4.1 Búsqueda secuencial

Búsqueda secuencial es la solución trivial para buscar en una colección de elementos. Cuando la colección está desordenada, es la única opción. Revisa/procesa cada elemento hasta encontrar el elemento deseado.

- En el peor caso, cada elemento de la colección es comparado contra la llave de búsqueda
- Si hay un *match*, la búsqueda termina y se retorna el índice del elemento
- Si no hay *match*, se retorna -1 (u otro índice negativo)

Por ejemplo, dado el siguiente array, para buscar el elemento 20, se seguiría el siguiente proceso:



4.1.1 Implementación

```
public class SequentialSearch {
    public static int search(int[] arr, int x) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == x) {
                return i;
            }
        }
        return -1;
    }
}
```

4.1.2 Time complexity

- Worst-case: $O(n)$
- Best-case: $O(1)$

4.1.3 Consideraciones importantes

- Es ineficiente para colecciones grandes
- Si el array no está ordenado o se quiere evitar ordenar, es la única opción

4.2 Búsqueda binaria

Búsqueda binaria es un algoritmo de búsqueda que encuentra la posición de un valor en un arreglo ordenado. A diferencia de la búsqueda lineal, que recorre el arreglo desde el primer elemento hasta el último, la búsqueda binaria divide el arreglo en dos mitades y compara el valor buscado con el elemento en el medio. - Si el valor buscado es menor que el elemento en el medio, la búsqueda continúa en la mitad izquierda del arreglo. - Si el valor buscado es mayor que el elemento en el medio, la búsqueda continúa en la mitad derecha del arreglo.

Este proceso se repite hasta que el valor buscado sea encontrado o hasta que el subarreglo de búsqueda sea vacío.

4.2.1 Ejecución de búsqueda binaria

Dado el siguiente arreglo:

Indices	0	1	2	3	4	5	6	7	8	
Elementos	1	2	3	4	5	6	7	8	9	

Para buscar el número 9: - Comenzamos comparando el número 9 con el elemento en el medio del arreglo, que es 5. - Como 9 es mayor que 5, la búsqueda continúa en la mitad derecha del arreglo. - Ahora comparamos el número 9 con el elemento en el medio de la mitad derecha del arreglo, que es 8. - Como 9 es mayor que 8, la

búsqueda continúa en la mitad derecha de la mitad derecha del arreglo. - Ahora comparamos el número 9 con el elemento en el medio de la mitad derecha de la mitad derecha del arreglo, que es 9. - Como 9 es igual a 9, hemos encontrado el número que buscábamos.

Visualmente, se puede representar de la siguiente manera:

Indices	0	1	2	3	4	5	6	7	8	
Elementos	1	2	3	4	5	6	7	8	9	

5 < 9						^				

7 < 9							^			

8 < 9								^		

9 == 9									^	

4.2.2 Implementación en Java

```
public static int binarySearch(int[] arr, int target) {
    int left = 0;
    int right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid;
        }

        if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
}
```

4.3 Búsqueda por interpolación

- Es una mejora sobre `búsqueda binaria`, específicamente si los valores en el array están distribuidos uniformemente. La búsqueda por interpolación calcula la posición de la mitad del array basado en el valor del elemento

buscado y los valores en los extremos del array.

- Búsqueda binaria siempre compara contra el elemento central del array, mientras que búsqueda por interpolación considera la llave de búsqueda antes de decidir contra cual elemento del array comparar.
- El código es igual al de búsqueda binaria, con la diferencia de que la posición del elemento medio se calcula de manera diferente:

```
mid = low + ((high - low) / (arr[high] - arr[low])) * (x - arr[low])
```

Low y high se refieren a los índices del array, y arr[low] y arr[high] a los valores en esos índices.

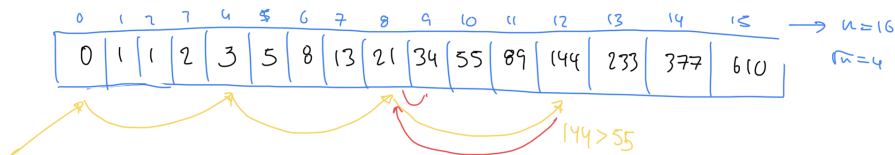
La mejora en rendimiento con respecto a búsqueda binaria se da en el caso promedio (que depende de la distribución uniforme de los elementos en el array), con una complejidad de tiempo de $O(\log \log n)$.

4.3.1 Referencias

- <https://iq.opengenus.org/time-complexity-of-interpolation-search/>

4.4 Búsqueda por salto (Jump Search)

- Aplicable para arrays ordenados
- Compara menos elementos que búsqueda lineal saltándose n elementos a la vez
- Determinar el tamaño del bloque de saltos es crucial para el rendimiento del algoritmo
- Normalmente se utiliza $\sqrt{n}$ como tamaño de bloque, donde n es el tamaño del array



4.4.1 Complejidad

Complejidad temporal: $O(\sqrt{n})$

4.4.2 Referencias

- <https://www.geeksforgeeks.org/jump-search/>

4.5 Pathfinding

Pathfinding se refiere a búsqueda de caminos entre dos puntos en un plano, especialmente útil para video juegos y simulaciones. Incluye una amplia variedad de algoritmos y no hay una solución única para todos los casos. Por ejemplo, la elección del algoritmo depende de:

- ¿Es el destino estacionario o móvil?
- ¿Hay obstáculos en el mapa?
- ¿Hay diferente tipos de terreno en el mapa?

4.5.1 Pathfinding básico

4.5.2 Pathfinding basado en grafos

4.5.2.1 Dijkstra

Cuando hay costos de movimiento según la dirección. Se lleva el costo acumulado de llegar a cada nodo y se elige el camino con menor costo.

```
frontier = PriorityQueue()
frontier.put(start, 0)
came_from = {}
cost_so_far = {}
came_from[start] = None
cost_so_far[start] = 0
while not frontier.empty():
    current = frontier.get()
    if current == goal:
        break
    for next in graph.neighbors(current):
        new_cost = cost_so_far[current] + graph.cost(current, next)
        if next not in cost_so_far or new_cost < cost_so_far[next]:
            cost_so_far[next] = new_cost
            priority = new_cost
            frontier.put(next, priority)
            came_from[next] = current
}
```

Visualmente, se puede entender la diferencia entre BFS y Dijkstra en el siguiente gráfico: <https://www.redblobgames.com/pathfinding/a-star/introduction.html#breadth-first-search>

4.5.2.2 A*

Considera el costo real (similar a Dijkstra) y una heurística que estima el costo restante. La heurística es una función que estima el costo de llegar al destino desde un nodo dado. La heurística debe ser admisible, es decir, nunca sobreestimar el costo real.

Se utiliza con ciertas mejoras, en game engines modernos, como Unity.

```
frontier = PriorityQueue()
frontier.put(start, 0)
came_from = {}
cost_so_far = {}
came_from[start] = None
cost_so_far[start] = 0
while not frontier.empty():
    current = frontier.get()
    if current == goal:
        break
    for next in graph.neighbors(current):
        new_cost = cost_so_far[current] + graph.cost(current, next)
        if next not in cost_so_far or new_cost < cost_so_far[next]:
            cost_so_far[next] = new_cost
            priority = new_cost + heuristic(goal, next)
            frontier.put(next, priority)
            came_from[next] = current
}
```

Para calcular la heurística, se puede utilizar la distancia Manhattan o la distancia Euclidiana. La distancia Manhattan es la suma de las diferencias en las coordenadas x y y, mientras que la distancia Euclidiana es la distancia en línea recta.

```
// Manhattan distance
function heuristic(a, b) {
    return abs(a.x - b.x) + abs(a.y - b.y)
}
```

Dependiendo del escenario, la heurística puede ser precalculada y almacenada en una tabla de búsqueda.

A* y Dijkstra son similares, pero A* es más rápido porque la heurística guía la búsqueda hacia el destino. La visualización aquí permite compararlas.

Aunque A* y Dijkstra encuentran el camino, la cantidad de comparaciones realizadas por A* es mucho menor. En el peor caso, A* es igual a Dijkstra, pero

en el mejor caso, A^* es mucho más rápido. e

4.5.3 Referencias

- Pathfinding
- Pathfinding on Grids

Chapter 5

Estructuras de almacenamiento externo

La jerarquía de memoria, se puede representar con la siguiente pirámide:

Conforme se “sube” en la jerarquía, la cantidad de memoria disminuye pero el costo y velocidad de la misma aumentan. Conforme se desciende, el costo y velocidad disminuyen pero la cantidad de memoria aumenta. Es por eso, que el cache, es una memoria muy rápida pero de poca capacidad, mientras que el disco duro es una memoria lenta pero de gran capacidad.

5.1 Conceptos esenciales

- **Dato:** Sucesión de símbolos representados con números o letras. No contienen ninguna información en sí mismo, sino que representan un *hecho* con algún significado dado por una unidad y su magnitud. Requieren organización y procesamiento para extraer información. Por ejemplo: *\$10*, *20 años*, *3 kg*, *10 metros*, *22 km*.
- **Información:** Es la interpretación significativa de los datos
- **Sistemas de almacenamiento:** Sistemas para almacenar datos en formato binario sin importar la información que se pueda extraer de dichos datos. Solo ven bloques crudos de bits.

5.2 Discos Duros (Hard-disk drives)

No deben confundirse con discos de estado sólido (SSD).

Son dispositivos de almacenameinto persistente que mantiene la información almacenada incluso cuando no están siendo alimentados con electricidad.

- Los datos se almacenan mediante magnetización de partículas dentro del material magnético de los discos.
- El disco es capaz leer los datos al detectar los patrones de magnetización creados al escribir los datos.

5.2.1 Componentes

- *Platos*: Discos circulares de material magnético en ambas caras
- *Eje*: sostiene uno o más platos conectados a un motor que gira los platos a un RPM constante.
- *Brazo actuador*: Mueve las cabezas de lectura/escritura a lo largo de los platos.
- *Cabezales*: Dispositivos electromagnéticos que leen y escriben datos en los platos.
- *Hard disk assembly*: Combinación de los platos, eje, brazo actuador y cabezales.

5.2.2 Funcionamiento

La velocidad del giro inflencia directamente a la velocidad de I/O. A mayor velocidad, mayor consumo energético y mayor costo.

- Velocidades para uso general (consumidor):
 - 5400 RPM
 - 7200 RPM
- Velocidades para uso profesional (servidores):
 - 10000 RPM
 - 15000 RPM

Los datos se escriben en círculos concentricos llamados *pistas* y se dividen en *sectores*. Un sector es la unidad mínima de almacenamiento en un disco duro y generalmente tiene un tamaño de 512 bytes.

5.2.3 Tiempos de acceso

El tiempo de búsqueda (seek time) es el tiempo que tarda el brazo actuador en moverse a la pista deseada (8 - 10ms en discos de 7200 RPM). El tiempo de latencia es el tiempo que tarda el sector deseado en pasar por debajo de la cabeza de lectura/escritura (4 - 5ms en discos de 7200 RPM).

El tiempo de acceso total es la suma del tiempo de búsqueda y el tiempo de latencia.

5.2.4 Tiempo de transferencia

Los discos duros tienen un tiempo de transferencia que es el tiempo que tarda en leer o escribir un bloque de datos. Este tiempo depende de la velocidad de rotación del disco y de la densidad de los datos. Por ejemplo, un disco de 7200 RPM tiene un tiempo de transferencia de 0.5ms.

El external data rate es la cantidad de datos que se pueden transferir por segundo. Por ejemplo, un disco de 7200 RPM con un external data rate de 300MB/s puede transferir 300MB de datos por segundo.

5.2.5 Interfaces

El HDD se conecta a otros componentes de la computadora a través de una interfaz. Las interfaces más comunes son:

- PATA (Parallel Advanced Technology Attachment)
 - Interfaz de 40 pines que se conecta a la placa madre.
 - Velocidad de transferencia de 133MB/s.
 - No es hot-swappable.
 - No se usa en la actualidad.
 - Solo para discos internos
- SATA (Serial Advanced Technology Attachment)
 - Evolución de PATA.
 - Estandar en computación moderna.
 - Velocidad de transferencia de 600MB/s.
 - Bajo consumo energético.
 - Hot-swappable.
- SCSI (Small Computer System Interface)
 - Interfaz de alta velocidad.
 - Se usa en servidores y estaciones de trabajo.
 - Hot-swappable.
- SAS (Serial Attached SCSI)
 - Evolución de SCSI.
 - Velocidad de transferencia de 12GB/s.
 - Hot-swappable.
 - Se usa en servidores y estaciones de trabajo.

5.3 Discos de Estado Sólido (SSD)

No son discos duros, sino dispositivos de almacenamiento de estado sólido que utilizan memoria flash para almacenar datos. No tienen partes móviles, lo que los hace más rápidos y menos propensos a fallas que los discos duros.

- Utiliza memoria flash para almacenar datos.
- Menor acceso aleatorio que los discos duros.
- No tiene latencia de búsqueda.
- Writes limitados (menor tiempo de vida útil).
- El precio es más alto que los discos duros.

5.3.1 Componentes

- **Controlador:** Procesador que ejecuta el firmware y es responsable de la gestión de la memoria, la corrección de errores y la interfaz con el sistema operativo.
- **Memoria NAND:** Memoria flash que almacena los datos. Se divide en celdas que pueden ser de tipo SLC, MLC, TLC o QLC.
- **DRAM:** Memoria volátil que se utiliza para almacenar datos temporales y mejorar el rendimiento.
- **Firmware:** Software que controla el funcionamiento del SSD.
- **Conector:** Interfaz que conecta el SSD a la placa madre. Los conectores más comunes son SATA y PCIE (utilizando el protocolo NVMe).
- **Form factor:** Tamaño y forma del SSD. Los form factors más comunes son 2.5", M.2 y U.2. También existen SSDs en formato de tarjeta de expansión PCIE.

5.3.2 Desempeño

La transferencia de datos de un SSD es mucho más rápida que la de un disco duro. Los SSDs modernos pueden alcanzar velocidades de lectura de hasta 550 MB/s y velocidades de escritura de hasta 520 MB/s. Un SSD PCIE NVMe puede alcanzar velocidades de lectura de hasta 3500 MB/s y velocidades de escritura de hasta 3300 MB/s.

5.4 Particionamiento de discos

Un solo disco físico puede ser dividido en varias particiones, cada una de las cuales se comporta como un disco independiente (lógico). Las particiones se pueden formatear con diferentes sistemas de archivos y se pueden montar en diferentes puntos de montaje.

En enlace entre lo físico y lo lógico se hace a través de la *metadada* almacenada en la tabla de particiones. La tabla de particiones es un registro que contiene información sobre las particiones del disco, como su tamaño, ubicación y tipo de

sistema de archivos. Esta tabla se encuentra en el primer sector del disco (MBR) y es leída por el sistema operativo al arrancar.

Una partición puede existir sin inicializarse.

5.4.1 Volumen

Es una partición inicializada con un *sistema de archivos*. Es la interfaz lógica que utiliza el sistema operativo para acceder a los datos almacenados en el disco.

5.5 Sistemas de archivos

- Forma en que los archivos son nombrados, organizados y almacenados en el disco.
- Es la interfaz lógica entre usuario y la capa física de almacenamiento.
- Almacena metadata/atributos de los archivos. Por ejemplo el nombre, tamaño, fecha de creación, etc.
- Fuertemente relacionado con el sistema operativo seleccionado. Por ejemplo, Windows utiliza NTFS, Linux utiliza ext4, etc.

El sistema de archivos requiere espacio en disco para almacenar la metadata. Por ejemplo, si se tiene un archivo de 1 KB, el sistema de archivos necesita espacio adicional para almacenar la metadata del archivo (nombre, tamaño, fecha de creación, etc.). Por tal motivo, posterior a formatear un disco, el espacio disponible es menor al espacio total del disco.

5.5.1 Propósito

- Nombrar y organizar archivos.
- Proveer un API para el acceso a los archivos: Creación, lectura, escritura, eliminación, etc.
- Almacenar un índice de archivos y directorios.
- Gestionar permisos y restricciones de acceso para mantener la integridad y seguridad de los datos.
- Funciones avanzadas como compresión, cifrado, cuotas de disco, etc.

5.6 Archivos

Se puede definir como el mecanismo de abstracción para hacer transparente al usuario, los detalles complejos del disco. Dicha abstracción provee un “canal de comunicación” para poder ejecutar operaciones como:

- Leer bytes
- Escribir bytes
- Desplazarse a una posición específica en el disco

Un archivo se puede procesar de varias formas:

- **Acceso secuencial:** byte por byte desde el principio hasta el final según el orden en que fueron escritos.
- **Acceso aleatorio:** acceso directo a cualquier parte del archivo (conocido como acceso directo) independientemente del orden de escritura.

Acceso secuencial es más rápido que el acceso aleatorio, pero el acceso aleatorio es más flexible.

- **Acceso indexado:** acceso a través de un índice que contiene la ubicación de los datos. Depende de la estructura y propósito del archivo. Por ejemplo, si es un archivo que contiene registros de clientes, un índice puede mapear el identificador de cada cliente, a la posición de su registro en el archivo. Precursor de las bases de datos.

5.6.1 Estructura de un archivo

Las estructuras más comunes de archivos son:

- Secuencia de bytes
- Registros de tamaño fijo
- Árboles de registros

5.6.1.1 Secuencia de bytes

En este caso, se trata de archivos planos que contienen una secuencia de bytes. No tienen estructura interna y se pueden leer o escribir byte por byte. El sistema operativo no tiene conocimiento de la estructura interna del archivo, por lo que no puede realizar operaciones avanzadas como búsqueda o indexación.

Los archivos se reducen a ser “cubetas” de bytes, maximizando la flexibilidad. Windows/Linux/macOS utilizan este tipo de archivos.

5.6.1.2 Registros de tamaño fijo

En este caso, el archivo se organiza en registros de tamaño fijo. Cada registro contiene una estructura de datos con campos de tamaño fijo. Los registros se pueden leer o escribir de forma secuencial o aleatoria. Este tipo de archivos permite realizar operaciones de búsqueda y ordenamiento.

Al crear un archivo de registros de tamaño fijo, se debe especificar la estructura de cada registro. Por ejemplo, si se tiene un archivo de registros de empleados, cada registro puede contener los campos: nombre, apellido, edad, salario, etc.

Muy utilizados en mainframes y bases de datos.

5.6.1.3 Árboles de registros

Similar al caso anterior, pero en lugar de almacenar los registros de forma secuencial, se almacenan en una estructura de árbol. Cada nodo del árbol

contiene un registro y apunta a otros nodos. Este tipo de archivos permite realizar operaciones de búsqueda y ordenamiento de forma eficiente.

5.6.2 Tipos de archivos

- **Archivos regulares:** Contienen datos de usuario. Pueden ser de texto o binarios.
- **Directorios:** Archivos que contienen una lista de archivos y directorios. Son partes esenciales del sistema de archivos.
- **Character special files:** Representan dispositivos de caracteres, como terminales y puertos serie.
- **Block special files:** Representan dispositivos de bloques, como discos duros y unidades flash.
- **Sockets:** Archivos que permiten la comunicación entre procesos.
- **Pipes:** Archivos que permiten la comunicación entre procesos.
- **Symbolic links:** Archivos que apuntan a otros archivos.
- **Binary files:** Archivos que contienen datos binarios, como imágenes, videos, etc.

5.7 Cache

- El cache se basa en el principio de localidad, que establece que los programas tienden a acceder a un conjunto reducido de direcciones de memoria en un corto periodo de tiempo.
- El concepto de caching puede ser aplicado a cualquier capa de computación, como CPU, disco, red, etc
- Se puede aplicar en distintas capas de la arquitectura de un sistema de información.
- Busca mantener datos usados frecuentemente en una ubicación óptima: cercano al usuario, cercano a la aplicación, en memoria más rápida, etc.

5.7.1 Funcionamiento general

1. Se solicita un dato
2. Se busca en el cache
3. Si se encuentra, se devuelve al usuario (cache hit)
4. Si no se encuentra, se busca en la fuente original (cache miss) y se almacena en el cache para futuras solicitudes.

5.7.2 Operaciones en el cache

Dado que la capacidad de un caché es limitada, usarlo de forma eficiente es crucial. Las operaciones más comunes son:

- **Hit:** Cuando la información solicitada se encuentra en el caché.
- **Miss:** Cuando la información solicitada no se encuentra en el caché.

Cada vez que se produce un *miss*, se debe decidir qué información se debe eliminar del caché para hacer espacio para la nueva información. Existen diferentes políticas de reemplazo, como *Least Recently Used* (LRU), *First In First Out* (FIFO), *Least Frequently Used* (LFU), *Most Recently Used* (MRU), etc. Veamos algunas de estas

5.7.2.1 Least Recently Used (LRU)

La política LRU reemplaza la entrada que no ha sido utilizada por más tiempo. Es la política más común y se basa en la idea de que si un elemento no ha sido utilizado recientemente, es menos probable que se utilice en el futuro.

5.7.2.2 First In First Out (FIFO)

La política FIFO reemplaza la entrada que ha estado en el caché por más tiempo. Es la política más simple y se basa en la idea de que los elementos que han estado en el caché por más tiempo son menos probables de ser utilizados en el futuro.

5.7.2.3 Least Frequently Used (LFU)

La política LFU reemplaza la entrada que ha sido utilizada menos veces. Se basa en la idea de que los elementos que han sido utilizados menos veces son menos probables de ser utilizados en el futuro.

5.7.2.4 Most Recently Used (MRU)

La política MRU reemplaza la entrada que ha sido utilizada más recientemente. Se basa en la idea de que los elementos que han sido utilizados más recientemente son más probables de ser utilizados en el futuro.

5.7.3 Tipos de cache a nivel general

5.7.3.1 En memoria

Se almacena en la memoria RAM. Se utiliza para almacenar datos que se acceden con frecuencia a nivel del proceso.

5.7.3.2 Caché de disco

Se almacena en el disco duro. Se utiliza para almacenar datos que se acceden con frecuencia a nivel del disco. Por ejemplo, los datos de un archivo que se accede con frecuencia.

5.7.3.3 Caché distribuido

Se almacena en varios servidores. Se utiliza para almacenar datos que se acceden con frecuencia a nivel de la red. Por ejemplo, los datos de un sitio web que se

accede con frecuencia.

5.7.4 Tipos de cache a nivel específicos

5.7.4.1 Application server cache

Es una especialización del cache en memoria. Se almacena en la memoria RAM del servidor de aplicaciones. Se utiliza para almacenar datos que se acceden con frecuencia a nivel de la aplicación.

5.7.4.2 Caché global

Se almacena en varios servidores. Se utiliza para almacenar datos que se acceden con frecuencia a nivel global. Por ejemplo, los datos de un sitio web que se accede con frecuencia en todo el mundo.

5.7.4.3 CDN (Content-delivery network)

5.7.5 Problemas que afectan a todos los caches

Chapter 6

Bases de Datos

Una **base de datos** es una colección organizada de información estructurada, o datos, almacenada típicamente de manera electrónica en un sistema informático. Está diseñada para permitir el acceso, gestión y actualización de datos de manera eficiente.

6.1 Conceptos Clave

- **Datos:** Cualquier información que puede ser almacenada electrónicamente.
- **Sistema de Gestión de Bases de Datos (DBMS):** Software que permite a los usuarios crear, leer, actualizar y eliminar datos en una base de datos.
- **Tablas:** Estructuras utilizadas para almacenar datos en una base de datos relacional. Las tablas constan de filas (registros) y columnas (campos).

6.2 Tipos de Bases de Datos

1. **Bases de Datos Relacionales:** Organizan los datos en tablas con relaciones predefinidas entre ellas.
2. **Bases de Datos NoSQL:** Diseñadas para manejar grandes volúmenes de datos no estructurados o semi-estructurados.
3. **Bases de Datos Orientadas a Objetos:** Almacenan datos como objetos en lugar de en tablas.

6.3 Terminología Clave de las Bases de Datos

- **Esquema:** Describe la estructura de la base de datos (tablas, campos, relaciones).
- **Clave Primaria:** Identificador único para cada registro en una tabla.

- **Clave Externa:** Campo en una tabla que hace referencia a la clave primaria en otra tabla.
- **Índice:** Mejora la velocidad de las operaciones de recuperación de datos en una tabla de la base de datos.

6.4 SQL (Structured Query Language)

- **SQL:** Lenguaje estándar para interactuar con bases de datos relacionales.
- **DDL (Data Definition Language):** Utilizado para definir y modificar estructuras de bases de datos (CREATE, ALTER, DROP).
- **DML (Data Manipulation Language):** Utilizado para manipular datos dentro de objetos de la base de datos (SELECT, INSERT, UPDATE, DELETE).

6.5 Diseño de Bases de Datos

- **Normalización:** Proceso de organización de datos en una base de datos para reducir la redundancia y la dependencia.
- **Diagramas ER (Entity-Relationship):** Representación visual de entidades de la base de datos y sus relaciones.

6.6 Beneficios de las Bases de Datos

- **Integridad de los Datos:** Asegura que los datos sean precisos y consistentes.
- **Control de Concurrencia:** Gestiona el acceso simultáneo a la base de datos.
- **Escalabilidad:** Capacidad para manejar grandes cantidades de datos.
- **Seguridad:** Protege los datos contra accesos no autorizados.

6.7 Introducción a SQL

SQL (Structured Query Language) es un lenguaje estándar utilizado para interactuar con bases de datos relacionales. Permite realizar diversas operaciones para gestionar datos de manera eficiente y precisa.

6.7.1 Operaciones Básicas en SQL

1. Consultas SELECT La operación más común en SQL es la consulta SELECT, que se utiliza para recuperar datos de una o más tablas.

Ejemplo:

```
SELECT columna1, columna2
FROM tabla
WHERE condición;
```

2. Inserción de Datos Para insertar nuevos registros en una tabla, se utiliza la instrucción INSERT.

Ejemplo:

```
INSERT INTO tabla (columna1, columna2)
VALUES (valor1, valor2);
```

3. Actualización de Datos Para actualizar registros existentes en una tabla, se utiliza la instrucción UPDATE.

Ejemplo:

```
UPDATE tabla
SET columna1 = nuevo_valor
WHERE condición;
```

4. Eliminación de Datos Para eliminar registros de una tabla, se utiliza la instrucción DELETE.

Ejemplo:

```
DELETE FROM tabla
WHERE condición;
```

Cláusulas y Expresiones SQL 1. Cláusula WHERE La cláusula WHERE se utiliza para filtrar registros basados en una condición específica.

Ejemplo:

```
SELECT columna1, columna2
FROM tabla
WHERE columna1 = 'valor';
```

2. Cláusula ORDER BY La cláusula ORDER BY se utiliza para ordenar los resultados de una consulta en orden ascendente o descendente.

Ejemplo:

```
SELECT columna1, columna2
FROM tabla
ORDER BY columna1 DESC;
```

3. Funciones Agregadas SQL proporciona funciones agregadas como COUNT, SUM, AVG, MIN y MAX para realizar cálculos en conjuntos de datos.

Ejemplo:

```
SELECT COUNT(*)
FROM tabla;
```

6.8 Introducción a NoSQL

NoSQL (Not Only SQL) es un término utilizado para describir bases de datos que no utilizan el modelo relacional tradicional basado en tablas. Estas bases de datos están diseñadas para manejar grandes volúmenes de datos no estructurados o semi-estructurados de manera flexible y escalable.

6.8.1 Características de NoSQL

- **Estructura Flexible:** Permite almacenar datos con estructuras flexibles sin necesidad de un esquema fijo.
- **Escalabilidad Horizontal:** Capacidad para manejar grandes volúmenes de datos distribuyendo la carga en múltiples servidores o nodos.
- **Modelos de Datos Diversos:** Soporta varios modelos de datos como documentos, grafos, columnas y clave-valor, optimizados para diferentes tipos de aplicaciones y cargas de trabajo.

6.8.2 Tipos de Bases de Datos NoSQL

1. **Bases de Datos de Documentos**
 - **Ejemplo:** MongoDB
 - **Características:** Almacena datos en documentos JSON o BSON. Es flexible y escalable.
2. **Bases de Datos de Grafos**
 - **Ejemplo:** Neo4j
 - **Características:** Modela datos como nodos y relaciones entre ellos. Útil para representar relaciones complejas.
3. **Bases de Datos de Columnas**
 - **Ejemplo:** Apache Cassandra
 - **Características:** Almacena datos en columnas en lugar de filas. Escalable y optimizado para escrituras rápidas.
4. **Bases de Datos Clave-Valor**
 - **Ejemplo:** Redis
 - **Características:** Almacena datos en pares clave-valor. Muy rápido y eficiente para almacenamiento en caché y sesiones.

6.8.3 Casos de Uso de NoSQL

- **Aplicaciones Web Escalables:** Ideal para aplicaciones web que requieren escalabilidad horizontal y manejo eficiente de grandes volúmenes de datos.
- **Análisis de Datos en Tiempo Real:** Utilizado en bases de datos como Apache Kafka o Elasticsearch para análisis de datos en tiempo real y búsqueda de texto completo.

6.8.4 Consideraciones y Limitaciones

- **Consistencia:** Algunas bases de datos NoSQL pueden sacrificar consistencia eventualmente consistente.
- **Herramientas y Ecosistema:** Aunque cada vez más robusto, el ecosistema y las herramientas de NoSQL pueden ser menos maduras en comparación con las bases de datos relacionales establecidas.

6.8.5 Ventajas de las bases de datos NoSQL

- **Sintaxis más flexible:** A diferencia de SQL, que tiene una sintaxis rígida, las bases de datos NoSQL permiten una mayor libertad en la estructura de los datos.
- **Escalabilidad horizontal:** Pueden crecer fácilmente agregando más servidores.
- **Rendimiento:** Operan principalmente en memoria, lo que reduce los tiempos de lectura y escritura.

